# DIFFUSION MODELS FOR ENERGY TIME SERIES GENERATION

by

**Arne Lescrauwaet**

in partial fulfillment of the requirements for the degree of

**Master of Science**
in Artificial Intelligence

at the Open University, Faculty of Science
Master Artificial Intelligence
to be defended publicly on 11/07/2025 at 09:30 AM.

| | | |
|---|---|---|
| Student number: | 852617312 | |
| Course code: | IM9506 | |
| Thesis committee: | prof. dr. Stefano Bromuri, | Open University |
| | dr. Daniel Stanley Tan, | Open University |
| | Sheng Yu, | Open University |

# Contents

# List of Figures

# List of Tables

# 1 INTRODUCTION

Electrical smart meter data plays a pivotal role in enabling various downstream tasks such as electrical grid optimization, demand-side management, forecasting, and simulation. These applications are critical for improving grid efficiency, enhancing energy sustainability, and accommodating renewable energy sources. However, access to real-world smart meter data is heavily restricted due to stringent privacy regulations and security concerns, as this data can reveal sensitive information about individual households, such as occupancy patterns, appliance usage, and overall lifestyle habits. This lack of accessibility poses significant challenges for researchers, policymakers, and industry stakeholders who rely on accurate data to develop and validate innovative solutions. To address these limitations, synthetic smart meter data has emerged as a potential solution. Synthetic data aims to simulate realistic scenarios and evaluate models in a privacy-preserving manner.

Generative models like Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs) offer promising avenues for creating synthetic smart meter time series data. However, both approaches have significant drawbacks. GANs are prone to mode collapse, limiting the diversity of generated outputs and failing to represent the full richness of the training data. Furthermore, their adversarial training process makes them notoriously unstable, often leading to oscillations, divergence, and difficulties in achieving convergence [2, 6, 18]. VAEs, while offering a latent space representation, can be challenging to manipulate this latent space to generate specific features or variations. Additionally, VAE performance is highly dependent on the chosen prior distribution. A mismatch between the prior and the true data distribution can result in unrepresentative synthetic samples [1].

Diffusion models have emerged as a promising alternative, demonstrating strong performance in high-fidelity data generation across various domains. However, their optimal configuration for time series generation remains an open question. A key challenge lies in defining the most effective neural network architecture for the noise estimation process. Additionally, the integration of metadata, such as weather conditions and household appliance usage, requires an effective embedding strategy to ensure meaningful conditioning during data synthesis. While prior studies have explored diffusion models for generating synthetic net load data and incorporating metadata for energy meter simulations [5, 13, 22], a systematic evaluation of these approaches for time series data remains lacking.

While numerous implementations and configurations for the $\epsilon_\theta$ network in diffusion models exist, there is limited comparative analysis of their performance specifically for time series generation. Additionally, there appears to be no comprehensive evaluation of embedding mechanisms for conditioning data within diffusion models for the time series domain. To better understand the influence of the $\epsilon_\theta$ network, we propose a comparison of several commonly used network architectures, including those employed in the works of Fu et al., Zhang et al., and Sikder et al. [5, 16, 22]. To evaluate the impact of metadata embedding on data generation, we suggest a comparison of popular embedding methods, including Multilayer Perceptron, Transformer Encoder, Fast Fourier Transform, and spectrogram-based embeddings. Additionally, to identify the optimal feature combination for conditioning, we propose an analysis of the impact of various features on the generation process.

To guide this exploration, the following key research areas will be investigated to advance the understanding and performance of diffusion models in energy time series generation:

- Identification of the most effective neural network architecture for the reverse diffusion process, evaluating commonly used configurations in recent literature.
- Analysis of different embedding mechanisms and their ability to incorporate conditional metadata effectively during the generation process.
- Assessment of the influence of various conditioning variables—such as weather and customer-specific features—on the quality and realism of the synthetic data.

The dataset used for this thesis is an open-source dataset from the Customer-Led Network Revolution project. This project was one of the largest smart grid demonstration projects in the UK, engaging 13,000 electricity customers. This initiative generated valuable insights through real-world trials involving actual customers and live networks. The dataset is anonymised and disaggregated, containing electricity consumption and photovoltaic (PV) generation data for individual customers, along with associated details such as customer information, tariff structures, and the studied technologies. The data is available on the Customer-Led Network Revolution website. To expand on the conditioning possibilities, a dataset of historical weather from the Leicester area, where the trial took place, was provided by Centrica. The dataset comprises key weather variables at an hourly interval and spans the period 1979 – 2024 of which the relevant dates will be selected.

This thesis investigates the application of diffusion models for generating high-quality synthetic residential energy consumption data, with a focus on architectural design and conditioning strategies. The Transformer-based TransFusion model was empirically identified as the most effective architecture, achieving superior statistical fidelity and downstream utility. Importantly, the study highlights that generative model evaluation must go beyond visual or metric similarity and instead prioritize functional realism, especially for practical forecasting tasks.

Through a systematic evaluation of embedding mechanisms and conditioning variables, the research demonstrates that model performance is highly sensitive to design choices and context. Notably, conditioning with select domain-informed variables—such as humidity and statistical features—substantially improved data realism, affirming that more conditioning data is not necessarily better. The findings establish clear trade-offs between realism, fidelity, and utility, and emphasize the importance of purposeful feature selection.

The primary contributions of this work are:

- A comprehensive architectural comparison, establishing TransFusion as the leading diffusion model for energy time series.
- An in-depth evaluation of embedding strategies, revealing the critical impact of projector network architecture on model performance.
- A nuanced analysis of conditioning variables that identifies the influence of specific features on both realism and statistical fidelity.
- Reinforcement of the Train Synthetic, Test Real (TSTR) approach as a practical benchmark for evaluating synthetic data utility in real-world ML applications.

The structure of this thesis is as follows: Section 2 provides an overview of related work, including the advancements and limitations of generative models for time series data, with a focus on diffusion models and their innovative applications. Section 3 outlines the research questions and methodologies employed to address them, detailing the experimental setup, evaluation strategies, and exploring the open-source dataset. Section 4 presents the results from running our experiments, which is followed by the section 5 in which the results are discussed. Finally, Section 6 concludes the proposal by summarizing the objectives, anticipated contributions, and the potential impact of the research.

# 2

# Related Work

## 2.1. Deep Learning Foundations and Diffusion Models

The landscape of generative modeling and time series analysis, particularly concerning the techniques relevant to this thesis, is built upon several influential architectural and theoretical foundations. This subsection aims to provide the necessary background by reviewing four such cornerstones: Long Short-Term Memory (LSTM) networks [8], renowned for sequence modeling; Transformer architectures [19], which revolutionized attention mechanisms; U-Net models[15], widely adopted for image-to-image tasks and subsequently adapted for sequence data; and Denoising Diffusion Probabilistic Models (DDPMs), a powerful class of generative models. These descriptions will underpin the review of related works that follows.

### Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) networks were designed to overcome the challenges of learning long-term dependencies faced by traditional Recurrent Neural Networks (RNNs). Their specialized architecture is centered around memory cells, each maintaining an internal state via a Constant Error Carousel (CEC). The CEC, a linear unit with a fixed self-recurrent connection, allows information and error signals to flow effectively through the cell across many time steps without vanishing or exploding.

The flow of information into and out of this CEC, and thus the updating of the cell's state, is controlled by three adaptive, multiplicative gate units:

- An **input gate** regulates how much of the current input and previous hidden state is allowed to update the memory cell's internal state, essentially deciding when to let new information in.
- An **output gate** controls how much of the current memory cell's internal state is exposed or passed on to the next hidden state and ultimately to the network's output, determining when the stored information should be accessed or influence other units.
- A **forget gate** (a crucial component in modern LSTMs, though the original emphasis was more on input/output protection of the CEC) controls how much of the previous cell state is retained or discarded.

By learning to open and close these gates, the LSTM network can selectively write, read, and preserve information within its memory cells for extended durations. This gating mechanism enables LSTMs to capture and exploit long-range dependencies in sequential data far more effectively than simpler RNN architectures [8].

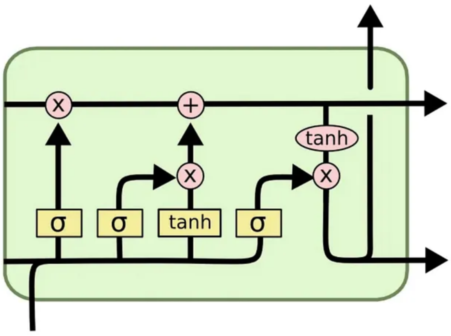



Figure 2.1: Standard LSTM Cell by KaNaung via Medium

### Attention Mechanism and Transformer Architecture

The attention mechanism and the Transformer architecture have become foundational in sequence processing, offering powerful alternatives to recurrent models.

**Attention Mechanism.** At its core, an attention mechanism allows a model to selectively weigh the importance of different parts of an input sequence when producing an output at a particular step, rather than relying solely on a compressed representation of the entire past (like the last hidden state of an RNN). It computes a context vector as a weighted sum of input features, where the weights, known as attention scores, reflect the relevance of each input part to the current computational focus.

**Transformer Architecture.** The Transformer architecture eschews recurrence entirely, relying instead on attention mechanisms. A key component is the **Scaled Dot-Product Attention**, which operates on three sets of learned vectors: Queries (Q), Keys (K), and Values (V).

- Queries represent the current context or element for which attention is being computed.
- Keys are associated with the input elements and are used, along with queries, to determine relevance through compatibility scores.
- Values are the actual representations of the input elements that are ultimately weighted and summed.

The process involves computing dot-product scores between a query and all keys, scaling these scores, applying a softmax function to obtain attention weights, and finally calculating the output as a weighted sum of the values.

This concept is extended in Transformers through **Multi-Head Attention**, where the attention mechanism is performed multiple times in parallel with different, learned linear projections of Q, K, and V. This allows the model to jointly attend to information from different representational subspaces.

The Transformer architecture typically consists of an **encoder** and a **decoder**, each a stack of identical layers.

- Each **encoder layer** comprises a Multi-Head Self-Attention sub-layer (where Q, K, V originate from the same sequence, allowing positions to attend to each other) and a position-wise Feed-Forward Network (FFN).
- Each **decoder layer** includes a Masked Multi-Head Self-Attention sub-layer (masked to prevent attending to future positions), a Multi-Head Encoder-Decoder Attention sub-layer (where Q comes from the decoder and K, V from the encoder output), and a position-wise FFN.

Both encoder and decoder layers employ residual connections around each sub-layer, followed by layer normalization. Since the Transformer lacks inherent sequential processing, **Positional Encodings** are added to the input embeddings to provide information about the relative or absolute position of tokens [19].

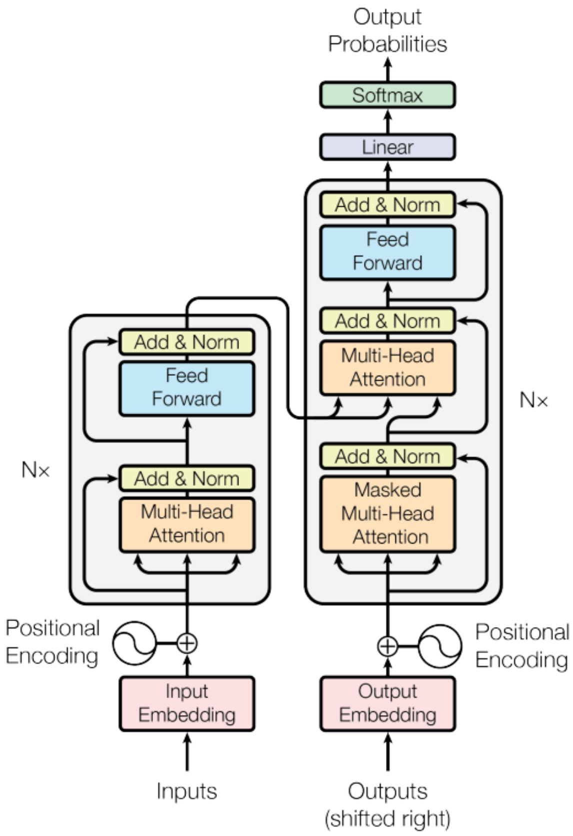



Figure 2.2: Default Transformer by [19]

### U-Net Architecture

The U-Net architecture, originally developed for biomedical image segmentation, is a convolutional neural network characterized by its symmetric U-shape, which efficiently combines feature extraction with precise localization.

**Core U-Net Architecture.** The U-Net consists of two main parts:

- A **contracting path (encoder)**: This path typically involves repeated blocks of convolutional layers (e.g., 3x3 convolutions with ReLU activations) followed by max pooling operations. This progressively downsamples the input, capturing hierarchical features and contextual information at increasingly coarser spatial scales, while the number of feature channels generally increases.

- An **expansive path (decoder)**: This path symmetrically upsamples the feature maps. Each upsampling step (e.g., using transposed convolutions or "up-convolutions") is typically followed by concatenation with high-resolution feature maps from the corresponding level in the contracting path via **skip connections**. These skip connections are a hallmark of U-Net, allowing the network to recover fine-grained spatial details lost during downsampling, which is crucial for precise localization. Further convolutions are then applied.

The final layers typically involve further convolutions (e.g., a 1x1 convolution) to map the features to the desired output format, such as a segmentation map.

**1D U-Net for Time Series.** When adapted for time series data, the U-Net architecture, often referred to as a 1D U-Net, replaces 2D operations with their 1D counterparts.

- **Encoder in 1D U-Net:** 1D convolutional layers slide along the time axis, extracting local temporal features. 1D pooling operations then reduce the length of the time series, capturing features at coarser temporal resolutions.

- **Decoder in 1D U-Net:** 1D upsampling layers increase the temporal resolution. Skip connections remain vital, transferring temporal details from the encoder to the decoder to aid in producing a precise, time-aligned output [15].

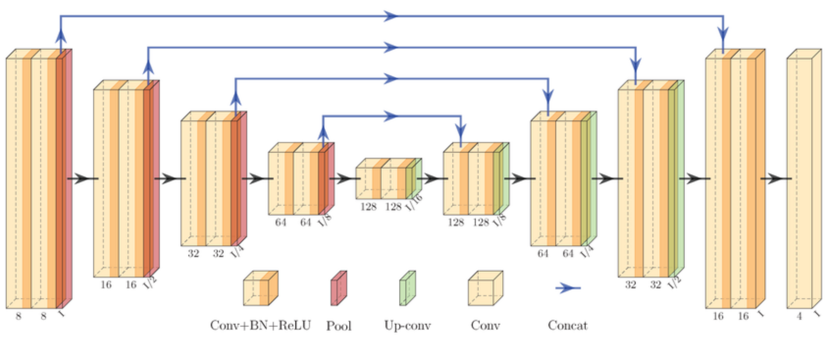



Figure 2.3: 1D Unet by [11]

### Denoising Diffusion Probabilistic Models (DDPMs)

Denoising Diffusion Probabilistic Models (DDPMs) are a class of generative models that learn to synthesize data by reversing a gradual, fixed noising process. The framework comprises two main processes:

- **Forward Diffusion Process (Fixed):** This process iteratively adds Gaussian noise to an initial data sample $x_0$ from the true data distribution over a sequence of $T$ timesteps. At each timestep $t$, a small amount of noise, scaled by a predefined variance schedule ($\beta_t$), is added to the data from the previous step ($x_{t-1}$) to produce $x_t$. After $T$ steps, $x_T$ approximates an isotropic Gaussian distribution (pure noise). This forward process is predetermined and does not involve learning. A key property is that the distribution of $x_t$ at any timestep $t$, given $x_0$, can be sampled directly in closed form.

- **Reverse Denoising Process (Learned):** The objective here is to learn to reverse the forward noising. Starting from pure noise $x_T \sim \mathcal{N}(0, \mathbf{I})$, the model iteratively denoises the sample to reconstruct a clean data sample $x_0$. This is achieved by training a neural network (often a U-Net like architecture) to predict the noise that was added at each step $t$ during the forward process. The network takes the noisy sample $x_t$ and the timestep $t$ (typically as a conditioning embedding) as input and aims to predict the noise component $\epsilon$ or, equivalently, the less noisy sample $x_{t-1}$ [7].

Training involves minimizing the difference between the actual noise added and the noise predicted by the network. DDPMs have demonstrated remarkable success in generating high-fidelity samples across various domains.

## 2.2. Synthetic Data Generation for Time Series

Synthetic data generation for time series is a well-established area, where advancements in machine learning have positioned generative models as a core approach. Among these, Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs) have become standard tools. Models such as TimeGAN, TimeVAE, and ConditionalGAN are regarded as state-of-the-art in this domain [3, 17, 20]. However, both architectures face inherent challenges. GAN-based methods are prone to training instability and mode collapse [2, 6, 18], which are particularly problematic when generating highly diverse time series data. VAEs, while offering more stable training and better distribution coverage, often produce outputs that lack sharpness and fidelity due to their probabilistic reconstruction framework. These limitations highlight the need for alternative approaches that can address these shortcomings while maintaining or improving the quality of the generated data.

## 2.3. Diffusion Models in Time Series

Diffusion models represent a newer generative modeling paradigm that has shown substantial promise, particularly in image and audio generation, and are now finding applications in the time series domain. Unlike GANs and VAEs, diffusion models generate data through a stepwise denoising process, progressively refining random noise into realistic samples. This iterative framework ensures stable training and avoids issues like model collapse that commonly plague GANs [4]. Despite their advantages, diffusion models have their drawbacks. A major limitation is their computational inefficiency, as their iterative nature requires significantly more time and resources during training and sampling [4, 10].

## 2.4. Innovations in Diffusion Models for Time Series

**Long-Sequence Generation with Diffusion Models** Sikder et al. introduce TransFusion, a novel model that combines diffusion models with transformers to generate long-sequence, high-fidelity time series data. Specifically, the transformer encoder is used in the backward diffusion process to capture long-term temporal dependencies through its attention mechanism. The model employs a cosine variance scheduler to gradually add noise to time-series data during the forward diffusion process, while the backward diffusion process iteratively denoises the data to approximate the original distribution. A significant innovation of TransFusion is its ability to generate time-series sequences with lengths up to 384, surpassing earlier methods limited to approximately 100 steps. While TransFusion offers high-quality data generation, its sampling process is computationally intensive, highlighting a potential area for future improvement [16].

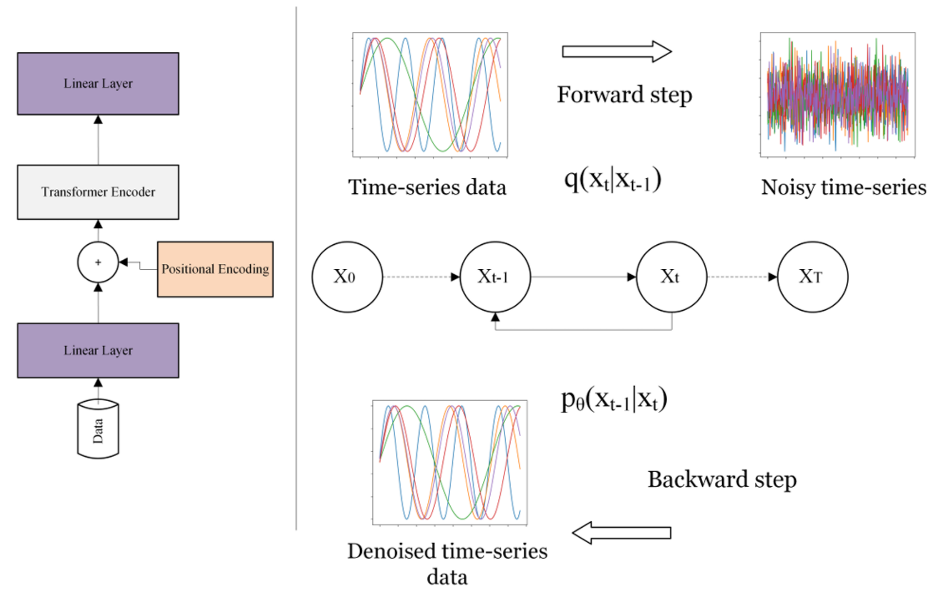



Figure 2.4: Diffusion model of the TransFusion paper [16]

**Meta-Driven Conditional Diffusion for Energy Data** Fu et al. propose a meta-driven conditional diffusion model for generating synthetic energy meter data that addresses key limitations in existing generative approaches. This model leverages metadata, including building type, meter type, and geographic location, to generate long-term, high-resolution energy consumption data. The integration of metadata allows for contextualized energy data generation that reflects real-world patterns. The model employs a U-Net architecture with a time-embedding layer to capture temporal dependencies in the energy data. While effective in generating long-term annual load profiles, the model's computational cost remains a limitation [5].

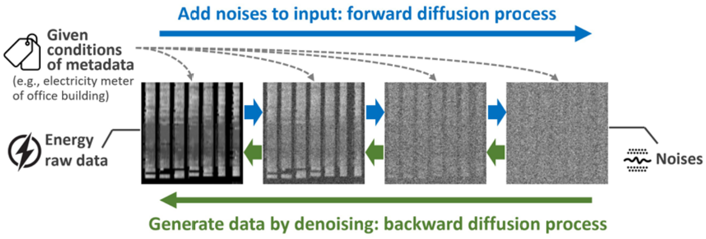



Figure 2.5: The meta-driven conditional diffusion model [5]

**Physics-Informed Diffusion for Net Load Data** Zhang et al. propose a physics-informed diffusion model (PDM) for generating synthetic net load data, integrating a Solar PV System Performance Model (PVSPM) into the denoising diffusion probabilistic model (DDPM). By combining a stochastic diffusion component with a deterministic physics-informed component, the model captures complex temporal dynamics while ensuring adherence to physical principles. The study highlights the value of embedding domain knowledge into generative frameworks, demonstrating the potential of physics-informed diffusion models for scenario-specific synthetic data generation [22].

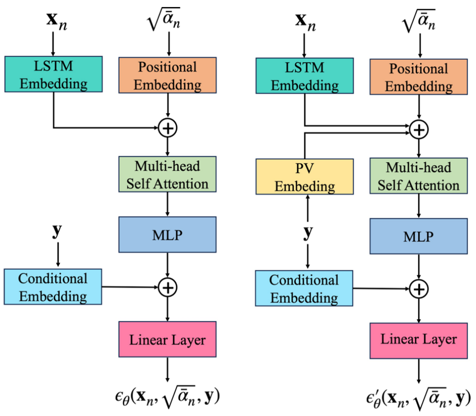



Figure 2.6: Baseline (left) and Physics informed (right) models. [22]

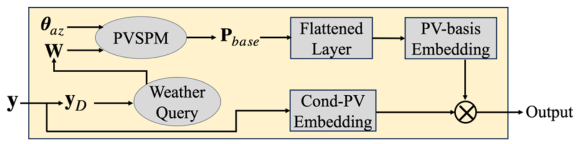



Figure 2.7: PV Embedding [22]

## 2.5. Application of Conditioning and Metadata in Diffusion Models

Diffusion models can incorporate additional data to guide the generation process through two primary approaches: conditioning and guidance. The first method, conditioned diffusion models, directly integrates contextual information, as demonstrated in [13]. The second approach, guidance, is used in frameworks like TS-Diff [21]. In the time series domain, conditioning is the predominant method, implemented by embedding metadata (e.g., building type, weather data) using techniques such as Multi-Layer Perceptrons (MLPs) or Recurrent Neural Networks (RNNs). For example, [13] utilized statistical properties of time series data for conditioning, while [5] incorporated metadata to ensure the generated data aligns with real-world scenarios. These approaches demonstrate the versatility and effectiveness of conditioning in improving synthetic data quality and relevance.

# 3

# Methodology

## 3.1. Datasets and Pre-processing

This section details the datasets employed in this study and the pre-processing pipeline developed to prepare them for model training. We utilize energy consumption data from the Customer-Led Network Revolution (CLNR) project and meteorological data for Leicester, UK.

### 3.1.1. Customer-Led Network Revolution (CLNR) Dataset

The primary dataset originates from the Customer-Led Network Revolution (CLNR) project[1], which collected smart meter readings from various domestic and SME customers in the UK and consists of the following four sub-datasets:

- Basic profiling of domestic smart meter customers (referred to as 'Domestic Baseline').
- Enhanced profiling of domestic customers with air source heat pumps ('Heat Pumps').
- Enhanced profiling of domestic customers with solar photovoltaics ('Solar PV').
- Enhanced profiling of domestic customers with electric vehicles ('EVs').

Across these datasets, measurements typically include a location identifier, timestamp, parameter description (e.g., consumption type), units (kWh), and the numerical consumption value. Specific characteristics of the utilized sub-datasets are outlined below.

#### Domestic Baseline Data

This dataset comprises 30-minute interval smart meter readings ('Electricity supply meter') for 8,798 unique domestic customers, recorded throughout 2012. After initial analysis, customers with excessive missing data were filtered out (details in Section 3.1.3), resulting in a cleaner subset for analysis. A histogram of the consumption values can be seen in Figure A.1.

[1] http://www.networkrevolution.co.uk/

#### Heat Pump Data

This dataset contains one-minute interval readings from 89 domestic households equipped with air source heat pumps, covering the period from May 1, 2013, to April 30, 2014. It includes measurements of whole-home power import and dedicated heat pump power consumption. No missing values were reported. However, the number of participating households was insufficient for use as standalone training data, and due to time constraints, the dataset was not integrated with the domestic baseline data. As a result, it was not included in the experiments.

#### Solar PV Data

This dataset comprises one-minute interval readings collected from 155 domestic households equipped with solar PV installations, spanning the period from June 1, 2012, to March 31, 2014. It includes measurements for whole-home power import and solar power generation. Due to a high number of customers with more than 5% missing data, this subset was excluded from the experiments.

#### EV Data

This dataset tracks energy usage in 144 households with electric vehicles, providing ten-minute interval readings from June 25, 2014, to March 31, 2015. It distinguishes between 'Charge point' consumption and overall 'House data'. Due to a high number of customers with more than 5% missing data, this subset was excluded from the experiments.

### 3.1.2. Leicester Weather Dataset

To incorporate environmental context, hourly meteorological data for Leicester, UK, was obtained[2]. While the original dataset spans several decades (January 1, 1979 – November 4, 2024), only the time period corresponding to the CLNR data utilized in this study was selected.

Key meteorological variables considered include:

- **Temperature**: `temp`, `feels_like`.
- **Humidity and Pressure**: `humidity`, `pressure`.
- **Wind Conditions**: `wind_speed`, `wind_deg`.
- **Precipitation**: `rain_1h`.
- **Cloudiness**: `clouds_all`.
- **Temporal Information**: Timestamps (`dt`, `dt_iso`) used for alignment with energy data.

Variables with substantial missing data (e.g., `visibility`, `sea_level`, `grnd_level`) or less relevance were excluded. Specific handling of remaining missing values (e.g., in `wind_gust`, `rain_3h`) is described in the processing stage. The `timezone` field, indicating daylight saving time shifts, was noted for temporal alignment.

[2]Data provided by Centrica plc

Table 3.1: Overview of Datasets Used in This Study

| Dataset | Time Interval | Customer Count | Key Features |
|---|---|---|---|
| **Domestic Baseline** | 30 minutes | 8,798 | Electricity supply meter readings; 2012; missing data filtered |
| **Heat Pump** | 1 minute | 89 | Whole home and heat pump power consumption; May 2013 – Apr 2014; no missing data |
| **Solar PV (photovoltaic)** | 1 minute | 155 | Whole home power import and solar generation; Jun 2012 – Mar 2014; filtered for missing data |
| **EV (Electric Vehicle)** | 10 minutes | 144 | Charge point vs whole home usage; Jun 2014 – Mar 2015; filtered subset used |
| **Leicester Weather** | 1 hour | – | Temp, humidity, pressure, wind, precipitation, cloudiness; aligned with CLNR time periods; partial variable exclusion due to missing data |

### 3.1.3. Data Pre-processing Pipeline

A multi-step pre-processing pipeline was implemented to prepare the datasets for time series modeling. The steps were applied in the following order, ensuring that operations like imputation and normalization didn't cause information leakage.

1. **Filtering (CLNR):** Time series (customers/meters) containing more than 5% missing values were removed from the respective datasets.

2. **Outlier Removal (CLNR):** Outliers within each time series were identified using the Interquartile Range (IQR) method. Values falling below $Q_1 - 1.5 \times IQR$ or above $Q_3 + 1.5 \times IQR$ were treated as missing values.

3. **Data Splitting:** The cleaned time series data (both CLNR and corresponding Weather data) was chronologically split into training (68%), validation (12%), and test (20%) sets.

4. **Imputation:** Missing values (resulting from original gaps or outlier removal) in the training, validation, and test sets were imputed using a combination of forward fill followed by backward fill. This ensures no future information is used for imputation within the training sequence implicitly.

5. **Normalization:** All numerical time series features (energy consumption, relevant weather variables) were scaled to a specific range [0 to 1 using Min-Max scaling]. The scaling parameters were calculated and applied per set.

6. **Temporal Feature Engineering:** Additional time-based features were extracted from the timestamps associated with the energy and weather data:

   - Standard features: day of week, hour of day, minute of day.
   - Cyclical Encoding: Features exhibiting cyclical patterns (e.g., day of week, hour, ...) were transformed using a sine function ($sin(2\pi \frac{x}{max(x)})$) to preserve their cyclical nature for the models.

7. **Final Feature Set:** The final feature set for input into the models consisted of the aggregated CLNR energy time series, relevant processed weather features, and the engineered temporal features. All features were appropriately scaled and aligned by timestamp.

## 3.2. Methodology for RQ1: Diffusion Model Effectiveness for Synthetic Energy Data

This section outlines the methodology employed to address the first research question: *Which neural network in diffusion models is the most effective for generating synthetic data in residential energy consumption?* We detail the foundational choices regarding the diffusion process and conditioning, describe the selected neural network architectures adapted from existing literature, and specify the training procedures and hyperparameters used for comparison.

### 3.2.1. Diffusion Process and Conditioning Approach

A core decision in designing diffusion models involves selecting the diffusion process itself. While both Denoising Diffusion Probabilistic Models (DDPM) and approaches based on Stochastic Differential Equations (SDE) are prevalent, this study adopts the DDPM framework. DDPM was chosen for its conceptual clarity, widespread adoption, and demonstrated strong empirical performance across various domains.

Effective generation often benefits from incorporating external information. Conditioning mechanisms, which directly input auxiliary data into the model during the reverse diffusion process, are common in time series applications [12]. While guidance strategies exist (e.g., TSDiff [12]), this work utilizes conditioning. Specifically, temporal information derived from the meter reading timestamps (as detailed in Section 3.1.3) serves as the conditioning input to guide the generation process.

### 3.2.2. Neural Network Architectures for Denoising

Within the DDPM framework, the denoising function $\epsilon_\theta(x_t, t, c)$ plays a central role in learning to reverse the forward diffusion process. To investigate the impact of neural architecture choices on this denoising process, we evaluate three distinct models, each adapted from recent diffusion and time series synthesis literature. Despite architectural differences, from transformers to recurrent and convolutional networks, all models operate within the same DDPM framework: they receive as input a noisy sample $x_t$, the diffusion timestep $t$, and optional conditioning information $c$, and output an estimate of the noise added at that step.

A common structural diagram (see Fig. 3.1) captures this unified process, where the encoder or latent processor differs per architecture, but each aims to minimize the denoising loss defined over the diffusion trajectory. The following subsections detail the specific adaptations and roles of each model within this shared framework.

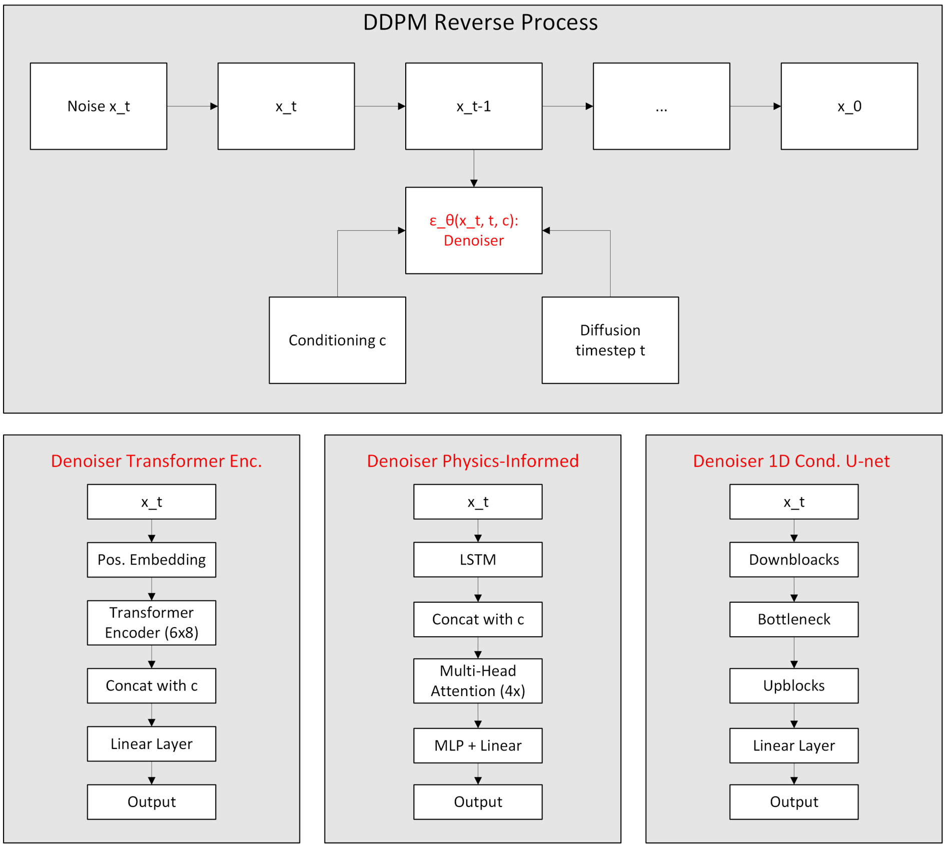



Figure 3.1: Common DDPM Framework with Denoising Network Architectures

### Transformer-Based Model (Adapted from TransFusion)

Adapted from Sikder et al.'s TransFusion architecture [16], which was originally designed for unconditional long-sequence generation, this model is restructured here to function as a conditional denoising network $\epsilon_\theta$ in DDPM. The input to the model is the noisy time series sample $x_t \in \mathbb{R}^{L\times d}$, where $L$ is the sequence length and $d$ the feature dimension, along with diffusion timestep $t$ and a conditioning vector $c$.

The conditioning features $c$ are processed through a two-layer MLP with Leaky ReLU activations, producing a latent embedding which is concatenated with the transformer's internal representations. This fusion occurs after the sequence is passed through 6 transformer encoder blocks. The model concludes with a linear projection head that outputs an estimate $\hat{\epsilon_\theta}$, the predicted noise to subtract during the reverse diffusion process.

This structure allows the model to leverage global temporal dependencies via attention while conditioning the generation process. Table 3.2 details the architecture's hyperparameters.

Table 3.2: Hyperparameters and Model Size for the Transfusion Model.

| Parameter Description | Value |
|---|---|
| Latent Dimension | 256 |
| Number of Layers | 6 |
| Number of Attention Heads | 8 |
| Learning Rate | $1 \times 10^{-4}$ |
| Optimizer Betas | (0.9, 0.99) |
| Number of Trainable Parameters | 11,342,417 |

#### LSTM-Attention Hybrid Model (Adapted from Physics-Informed Baseline)

This architecture is based on the baseline model proposed by Zhang et al. [22] for generating net load data using physics-informed diffusion models. Here, we retain only the data-driven baseline component to model $\epsilon\theta$. The model begins with an LSTM encoder that processes the input noisy sequence $x_t$, capturing sequential dependencies in the latent representation $h_t$. Positional embeddings and conditioning variables are processed via MLP layers and concatenated with the LSTM output. This combined embedding passes through a Multi-Head Self-Attention block to capture contextual dependencies, followed by a feedforward MLP layer.

The final layer concatenates the attention-augmented embedding with the conditioning vector again, ensuring the model can revise its prediction in light of external variables. A final linear projection outputs $\hat{\epsilon\theta}$. Table 3.3 contains key hyperparameters.

Table 3.3: Hyperparameters and Model Size for the Physics-Informed Model.

| Parameter Description | Value |
|---|---|
| Latent Dimension | 1000 |
| Number of Attention Heads | 4 |
| Learning Rate | $5 \times 16^{-4}$ |
| Decay Rate | 0.9 |
| Gamma | 0.9 |
| Number of Trainable Parameters | 20,717,001 |

#### 1D U-Net Based Model (Adapted from Conditional U-Net)

This model draws on the conditional U-Net approach introduced by Fu et al. [5], adapted here to operate on 1D time series data within a DDPM framework. The architecture directly estimates $\epsilon_\theta$ for a given noisy input $x_t$, timestep $t$, and conditioning vector $c$.

It consists of three main components:

- Encoder Path: A hierarchy of 1D convolutions and pooling operations compress the temporal dimension while increasing channel depth. The goal is to extract hierarchical temporal features from the input.

- Bottleneck: At the deepest level, the model receives not only the downsampled features but also the embedded timestep $t$ and conditioning vector $c$.

- Decoder Path: Symmetrical to the encoder, this path upsamples features using transposed convolutions. Skip connections from the encoder ensure fine-grained temporal details are retained. Conditioning embeddings and timestep information are injected into the upsampling blocks as well.

The model outputs $\hat{\epsilon_\theta}$ with the same dimensionality as the input $x_t$. Table 3.4 summarizes the model details.

Table 3.4: Hyperparameters and Model Size for the Unet Model.

| Parameter Description | Value |
|---|---|
| Learning Rate | $1 \times 10^{-4}$ |
| Number of Layers | 3 |
| Channel Multipliers | 1, 2, 4 |
| Embedding Dimension | 32 |
| Base Channels | 32 |
| Number of Trainable Parameters | 1,278,369 |

#### Architectural Comparison in DDPM Context

All three models serve as trainable estimators of the denoising function $\epsilon_\theta(x_t, t, c)$, differing mainly in how they structure temporal dependencies and integrate conditioning information:

- TransFusion (Transformer): Excels in modeling long-range dependencies and global attention. Conditioning is fused after the transformer layers.

- Physics-Informed (LSTM + Attention): Combines sequential memory (LSTM) with attention over hidden states. Conditioning is fused both before and after the attention layers.

- 1D U-Net: Utilizes hierarchical convolutional processing with skip connections and feature injection at multiple points, facilitating multi-scale feature learning.

### 3.2.3. DDPM Implementation Details

The specifics of the DDPM process also varied slightly between model types, primarily based on the implementations provided alongside the original architectures or common practices.

All models utilized 1000 diffusion timesteps. The Physics-Informed and Transfusion models employed a cosine noise schedule. In contrast, the 1D U-Net model used a linear schedule, with its $\beta$ values ranging from an initial $\beta_{\text{start}} = 10^{-4}$ to a final $\beta_{\text{end}} = 0.02$.

## 3.3. Methodology for RQ2: Impact of Embedding and Integration Mechanisms

This section details the methodology designed to investigate the second research question: *How do different embedding mechanisms impact the performance of diffusion models in generating or augmenting customer energy consumption profiles?* Specifically, we evaluate the influence of different techniques for embedding conditioning metadata and integrating these embeddings into the diffusion model.

### 3.3.1. Conditioning Embedding Mechanisms

To integrate auxiliary information into the denoising networks, several conditioning embedding strategies were explored. These embeddings encode external or contextual data—such as temporal metadata or structured side information—into a vector format suitable for fusion with the model's latent representations. Four embedding strategies were implemented and evaluated:

**1. MLP-Based Embedding**
This approach employs a simple Multi-Layer Perceptron (MLP) to transform tabular or structured metadata—such as calendar features or static building attributes—into dense embeddings. The MLP comprises two linear layers with a hidden dimension equal to the latent space $H$, and uses a LeakyReLU activation function between them with a negative slope of 0.01. Formally, given an input metadata vector $c \in \mathbb{R}^d$, the network outputs an embedding $h_c \in \mathbb{R}^H$, which is typically concatenated with or added to the model's internal representations. The fusion occurs at key stages, such as within attention inputs, LSTM outputs, or convolutional layers, depending on the backbone architecture.

**2. Transformer Encoder Embedding**
Inspired by the conditional embedding strategy from TransFusion [16], this method uses a transformer encoder to produce context-rich embeddings of conditioning metadata. The input vectors are first projected to match the model's hidden size and then passed through a transformer encoder consisting of 6 layers with 8 self-attention heads per layer. This setup enables the model to capture complex dependencies and internal structure within the metadata itself, leading to more expressive and context-aware conditioning representations.

**3. Spectrogram Embedding (STFT-Based)**
To condition on time-series inputs, a spectrogram-based embedding approach was used. Specifically, the Short-Time Fourier Transform (STFT) converts the 1D signal into a 2D time-frequency representation, enabling the model to capture both temporal dynamics and periodic structures. The STFT was configured with the following parameters: `n_fft = 256`, `hop_length = 64`, `power = 2.0`, `center = True`, and `pad_mode = reflect`. The resulting spectrogram is passed through a projection network—either a Multi-Layer Perceptron (MLP) 3.2 or a shallow Convolutional Neural Network (CNN)3.2, depending on the downstream architecture—to produce a compact embedding vector.

**4. Fast Fourier Transform (FFT) Embedding**

For an alternative frequency-domain embedding, a Fast Fourier Transform (FFT) was applied to the conditioning signals 3.2. The FFT yields complex-valued representations, and two encoding variants were implemented: one based on the magnitude spectrum alone, and another using both the real and imaginary components. The selected features are concatenated and processed by an MLP (structured identically to the one used in the MLP-based embedding) to produce the final embedding vector. This method is particularly efficient and useful when frequency-domain characteristics dominate the conditioning signal.

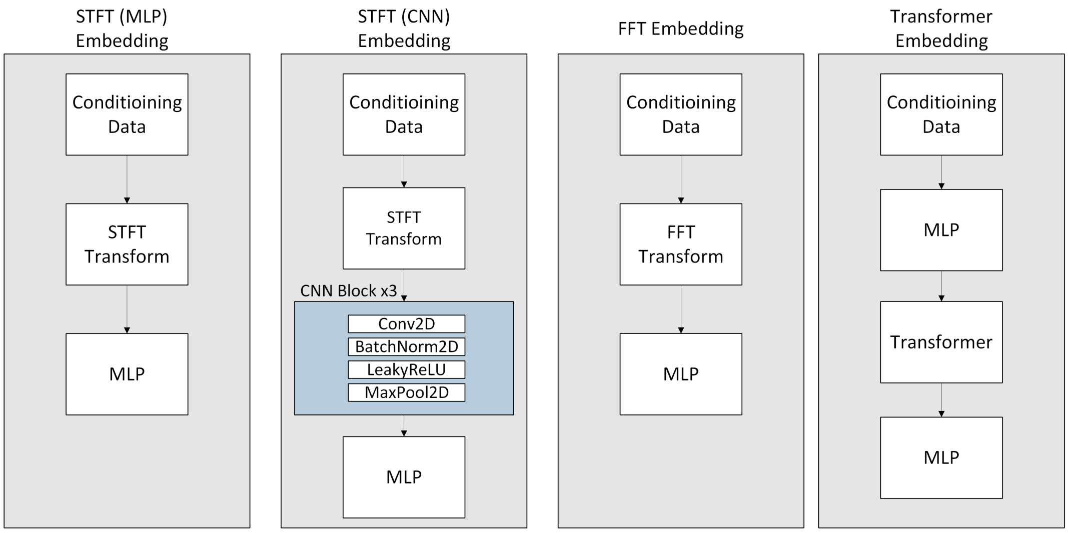



Figure 3.2: STFT (MLP) - STFT (CNN) - FFT - Transformer embedding

### 3.3.2. Conditioning Integration Strategies

In addition to the choice of embedding architecture, the method used to integrate conditioning information into the denoising network is a critical design decision. Two integration strategies are explored in this study:

**Concatenation.** This baseline method simply concatenates the conditioning embedding with the main model's latent representation (e.g., of the noisy time series) along the feature dimension. The combined representation is then passed through subsequent layers of the denoising network. This strategy is widely used due to its simplicity and low computational overhead, though it lacks explicit modeling of interactions between conditional and primary features.

**Cross-Attention.** To enable more expressive and dynamic conditioning, we implement a cross-attention mechanism inspired by transformer-based attention layers [9]. In this setup, the noisy time series representation serves as the query, while the conditioning embedding is treated as the key and value. Formally, we use a multi-head attention module with $H$ heads and latent dimensionality $d$:

- Inputs are first normalized using separate LayerNorm operations for the query and key/value streams.
- To improve numerical stability and prevent exploding gradients, extreme values are clipped to the range $[-100, 100]$ at several points in the forward pass.

- The residual attention output is scaled down (by a factor of 0.1) before being added back to the input query representation.
- A two-layer feedforward network with GELU activation and dropout is applied post-attention, again with scaled residuals and value clipping for stability.
- The final output is sanitized using `torch.nan_to_num` to remove any remaining NaNs or infinities.

This architecture allows the model to learn soft alignment patterns between latent representations of the noisy input and the conditioning signal, which may be especially useful when conditioning on structured or temporally rich auxiliary inputs.

### 3.3.3. Experimental Design

To systematically evaluate the impact of these mechanisms while managing computational resources, the following experimental design was adopted:

1. **Baseline Model Selection:** The best-performing diffusion model architecture identified in the evaluation for RQ1 (Section 3.2) is selected as the foundational architecture for all RQ2 experiments. Let this model be denoted as $M^*_{RQ1}$. $M^*_{RQ1}$ utilizes MLP-based embedding and concatenation as its default conditioning strategy.

2. **Embedding Mechanism Comparison:** To evaluate the impact of different embedding techniques, the following configurations are compared. Each embedding mechanism is implemented as described in Section 3.3.3, while the integration strategy is held constant using concatenation.

   - Configuration 1 (Baseline): $M^*_{RQ1}$ using its default MLP-based embedding mechanism.
   - Configuration 2: $M^*_{RQ1}$ where the default MLP embedder is replaced with a Transformer Encoder embedding mechanism.
   - Configuration 3: $M^*_{RQ1}$ where the default MLP embedder is replaced with a Spectrogram-based embedding mechanism (applied to relevant conditioning features).
   - Configuration 4: $M^*_{RQ1}$ where the default MLP embedder is replaced with a FastFourierTransform-based embedding mechanism (applied to relevant conditioning features).

   All other aspects of the model architecture and hyperparameters are kept identical to the baseline $M^*_{RQ1}$ configuration from RQ1 experiments.

3. **Integration Strategy Comparison:** The impact of the integration strategy is evaluated by comparing the configurations outlined in Section 3.3.2, while keeping the embedding mechanism fixed to the default MLP-based approach.

   - Configuration 1 (Baseline): $M^*_{RQ1}$ using MLP-based embedding and the default concatenation strategy.
   - Configuration 2: $M^*_{RQ1}$ using MLP-based embedding, but replacing the concatenation layer(s) with a cross-attention mechanism. This mechanism incorporates 3 cross-attention heads (representing half the number of self-attention heads in the presumed $M^*_{RQ1}$ architecture – adjust if baseline model differs).

Again, all other hyperparameters remain consistent with the $M^*_{RQ1}$ baseline.

This approach allows for isolating the effects of the embedding technique and the integration strategy relative to a high-performing baseline architecture. The evaluation metrics used will be consistent with those employed for RQ1. An overview of this approach can be seen in 3.3.

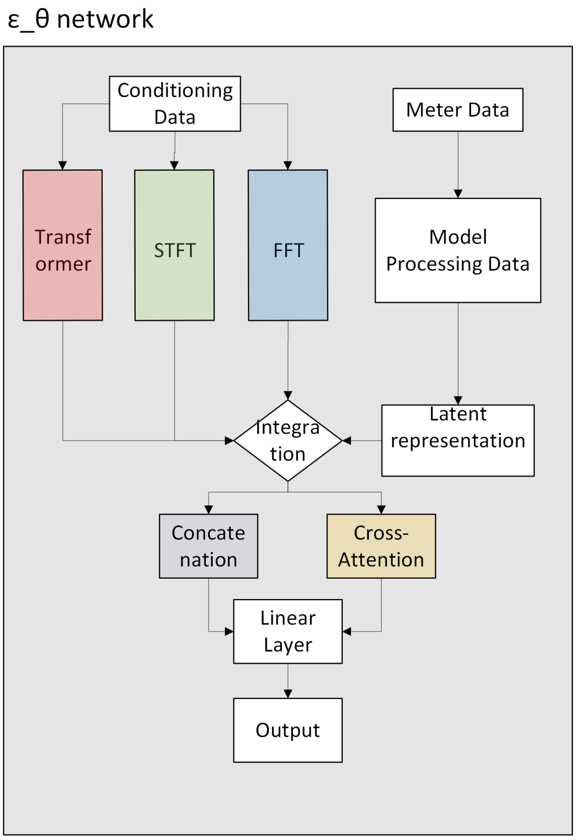



Figure 3.3: Block chart for Research Question 2 methodology

## 3.4. Methodology for RQ3: Impact of Variables in the Conditioning Data

This section outlines the methodology developed to address the third research question: *What is the impact of the variables in the conditioning data in conditioned diffusion models?* Specifically, we assess how the choice and grouping of conditioning variables influences model performance.

### 3.4.1. Conditioning Data

In previous experiments, conditioning was based solely on engineered time features. For this stage, we extend the conditioning space to include:

- External weather data from the Leicester Weather dataset.
- Statistical features derived from each meter's historical consumption over the sequence window.

### Time Feature Representation

Time-based conditioning features are derived per timestamp within each sequence and encoded as follows:

Table 3.5: Encoded Time-Based Conditioning Features

| Feature | Encoding Description |
|---|---|
| `minute_sin` | $\sin(2\pi \cdot \text{minute}/60)$ |
| `hour_sin` | $\sin(2\pi \cdot \text{hour}/24)$ |
| `day_sin` | $\sin(2\pi \cdot \text{day\_of\_week}/7)$ |
| `is_weekend` | Binary (1 if Saturday or Sunday, else 0) |

These features are generated per timestep, for each sequence in the dataset.

### Weather Features

The following weather variables are included from the Leicester dataset:

Table 3.6: Overview of Weather Features Used

| Feature Name | Description | Category |
|---|---|---|
| `temp` | Temperature | Temperature-related |
| `feels_like` | Perceived temperature | Temperature-related |
| `temp_min` | Minimum daily temperature | Temperature-related |
| `temp_max` | Maximum daily temperature | Temperature-related |
| `humidity` | Relative humidity | Humidity-related |
| `clouds_all` | Cloud coverage | Cloud-related |
| `dew_point` | Dew point | Other |
| `pressure` | Atmospheric pressure | Other |
| `wind_speed` | Wind speed | Other |
| `wind_deg` | Wind direction | Other |
| `wind_gust` | Wind gust speed | Other |

These weather values are aligned temporally with the meter sequences and injected per timestep, enabling fine-grained temporal conditioning.

### Statistical Features

In addition to timestamp-level features, statistical summaries are computed for each sequence of meter readings and injected as constant features across the entire sequence. These statistics are computed separately for the train, validation, and test sets:

- **Mean**
- **Standard Deviation**
- **Skewness**
- **Kurtosis**
- **Minimum**
- **Maximum**

These values summarize the full consumption profile across the sequence window (e.g., one week), and are held constant across all timesteps for a given sequence.

### 3.4.2. Evaluation Configurations

To assess the influence of different conditioning variables on model performance, we train separate diffusion models using distinct feature combinations. Each configuration includes the core time-based features and one additional group of conditioning variables:

- Time features combined with all weather features.
- Time features combined with statistical features.
- Time features combined with temperature-related features (`temp`, `feels_like`, `temp_min`, `temp_max`).
- Time features combined with the cloud-related feature (`clouds_all`).
- Time features combined with the humidity-related feature (`humidity`).

By isolating specific subsets of the conditioning data—particularly those previously identified as influential for energy modeling [14]—this design enables a controlled comparison of their individual effects. The goal is to quantify how each group contributes to the generation quality of the diffusion model under consistent training conditions. An overview of this experimental setup is shown in Figure 3.4.

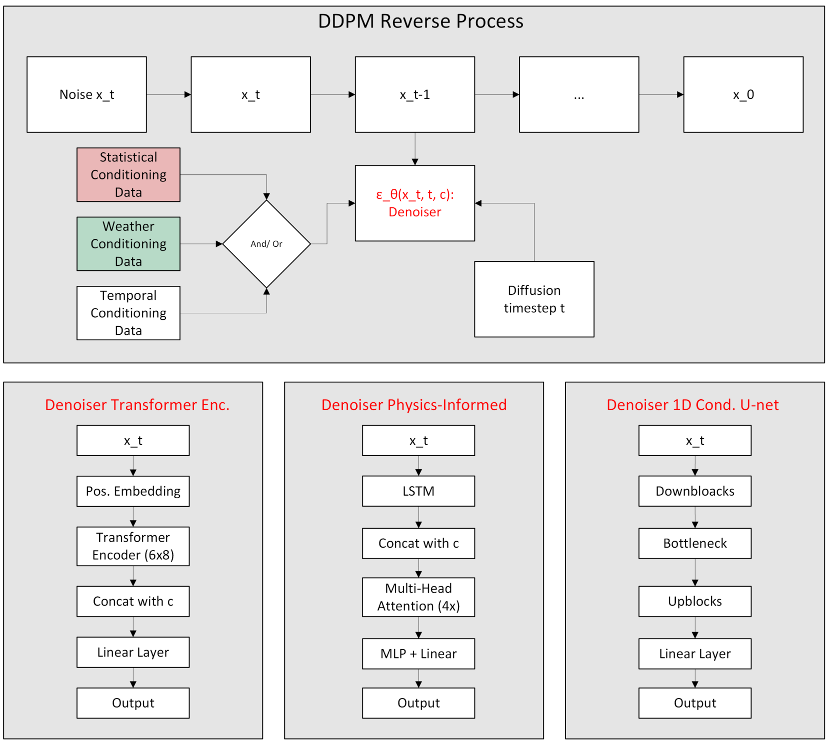



Figure 3.4: Block chart for Research Question 3 methodology

# 4

# Results

The performance of the generated models was assessed using a consistent set of metrics and evaluation procedures across all research questions. For each evaluation, the trained models generated synthetic samples conditioned on information derived from the test set. Both the real and generated sequences followed a consistent shape of `(batch_size, channels, sequence_length)`, typically `(128, 1, 336)`, corresponding to one week of hourly data.

## 4.1. Evaluation Methodology

The evaluation strategy integrates statistical, discriminative, predictive, and qualitative analyses. Each approach is outlined below:

1. **Feature Distribution Analysis.** To assess the statistical similarity between real and synthetic samples, a range of descriptive features was computed for each sequence. These included standard statistical metrics (mean, standard deviation, skewness, kurtosis, minimum, maximum) as well as temporal autocorrelation values at key lags (1, 2, 24, 48, 96, and 335). The distributions of these features were visualized using histograms and Kernel Density Estimation (KDE) plots, and quantitatively compared using the Wasserstein Distance.

   While this approach provides a useful summary of global statistical alignment, we found it to be a limited indicator of generation quality in our case. In particular, models that matched these distributions well did not always produce realistic or high-fidelity time series when examined directly. This suggests that these statistical measures, though informative, fail to fully capture the underlying time series distribution—especially in the presence of strong temporal structure and multimodal behavior.

   For this reason, we interpret the results of this analysis as only one aspect of model evaluation, and place greater emphasis on discriminative and downstream performance metrics in the subsequent sections.

2. **Discriminative Evaluation.** To assess the distinguishability of synthetic samples, a Transformer-based binary classifier was trained to differentiate between real and generated data. The classifier architecture consists of:

- An input linear projection layer mapping the input dimension ($d = 48$) to a hidden dimension ($H = 64$).
- A 2-layer Transformer encoder with 2 attention heads and positional self-attention over the sequence.
- Adaptive average pooling over the sequence dimension.
- A fully connected output layer producing logits over two classes (real vs. synthetic).

The classifier was trained using labeled data from the training set, where synthetic data was generated using training conditioning. After training, it was evaluated on test set data and newly generated synthetic samples. The goal was for classification accuracy to approach 50%, indicating indistinguishability between real and synthetic data.

3. **Train on Synthetic, Test on Real (TSTR).** To evaluate the practical utility of synthetic data, the TSTR paradigm was adopted. Forecasting models were trained on either real or synthetic training data and evaluated on a held-out real test set. For this purpose, an `XGBoost` regressor was used with the following configuration:

   - Objective: `reg:squarederror`
   - Number of trees: 100
   - Maximum depth: 5
   - Learning rate: 0.1
   - Random seed: 42

   This setup enables comparison of predictive generalization from synthetic data relative to real data, helping to quantify how well synthetic samples retain signal relevant for forecasting.

4. **Direct Data Comparison.** In addition to feature-level and model-based comparisons, direct visual inspection was conducted. Raw real and generated sequences were plotted using time series line graphs to assess qualitative fidelity and pattern replication.

## 4.2. RQ1: Which type of diffusion model is the most effective for generating synthetic data in residential energy consumption?

### 4.2.1. Evaluation of the TransFusion Model

The performance of the TransFusion model is evaluated using the four-part framework introduced in Section 4: feature distribution analysis, discriminative evaluation, TSTR evaluation, and direct data comparison.

#### Feature Distribution Analysis

The statistical characteristics of the generated time series were compared against those of the real data using both visual and quantitative analyses. Visualizations of the statistical feature distributions are presented in Figure 4.1, with generated data shown in red and real data in blue. A detailed qualitative analysis of these distributions reveals the following:

**Mean.** The agreement between the two distributions is very strong, as indicated by the low Wasserstein distance. Both central tendency and overall spread are well-replicated by the generated data. There's a slight tendency for the generated data to overestimate the density in the 0.18 to 0.22 range and slightly underestimate it around 0.15. However, overall, the model effectively captures the distribution of mean values.

**Standard Deviation.** The distributions are generally similar in shape. However, the central tendency of the generated data is shifted slightly to the left, with its peak occurring around 0.15, indicating that the model tends to generate data with slightly lower standard deviations on average compared to the real data. The peak of the generated distribution is also higher and somewhat narrower, and it less effectively captures the tail representing higher standard deviation values seen in the real data. Despite these subtle differences, the low Wasserstein distance suggests a reasonably good overall fidelity.

**Skewness.** The generated data approximates the location of the primary mode of the real data reasonably well. However, it fails to adequately capture the right-skewness and the presence of higher skewness values (the tail end) that are characteristic of the real data. The spread of the generated data is noticeably narrower, and it does not replicate the secondary mode or the extent of the tail observed in the real distribution. The peak density of the generated data's main mode is higher than that of the real data.

**Kurtosis.** There is a substantial discrepancy between the real and generated distributions for kurtosis, as highlighted by the relatively high Wasserstein distance. The generated data significantly underestimates the central tendency of the real data's kurtosis. Furthermore, it fails to capture the considerable spread and the long right tail of the real distribution, leading to a much narrower distribution that is shifted towards lower kurtosis values. The shapes of the two distributions are markedly different, with the generated data not reflecting the higher peak and extended tail of the real data.

**Minimum Value.** This feature shows the best agreement between the real and generated distributions, evidenced by the extremely low Wasserstein distance. The model accurately captures the sharp peak near zero and the subsequent rapid decay in density. The generated distribution effectively replicates the characteristics of the real data's minimum value distribution.

**Maximum Value.** While both distributions indicate a concentration at high maximum values, the generated data does not fully replicate the acute peak at the absolute maximum observed in the real data. The generated distribution tends to smooth this peak, resulting in a lower density at the 1.0 mark and a slightly wider dispersion towards lower maximum values (between 0.7 and 0.95).

Table 4.1: Wasserstein Distances for Statistical Features of the TransFusion Model Output

| Statistical Feature | Wasserstein Distance |
|---|---|
| Mean Value | 0.0070 |
| Minimum Value | 0.0025 |
| Maximum Value | 0.0494 |
| Standard Deviation | 0.0103 |
| Skewness | 0.2413 |
| Kurtosis | 0.8912 |

While the model effectively reproduces lower-order statistics (e.g., mean, standard deviation), its performance on higher-order moments like skewness and kurtosis is significantly weaker. As discussed in Section 4, this discrepancy highlights a key limitation of statistical feature matching: it does not always reflect overall fidelity in the time series domain, especially in cases where the real data is multimodal or contains diverse structural patterns. Despite low Wasserstein distances for some features, qualitative inspection shows that these metrics may underestimate the distributional complexity and structural mismatch.

### Temporal Feature Analysis

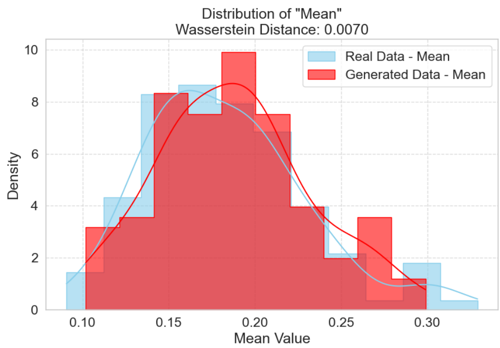
To evaluate the temporal dynamics of the generated data, autocorrelation values were computed at selected lags and compared to the corresponding values from real data. The results are visualized in Figure 4.2.

Overall, the model reproduces the temporal dependencies of the real data with reasonable accuracy. Across all lags—from short-term (lag 1 and 2) to longer-term (lag 335)—the distributions of autocorrelation values for the generated data closely mirror those of the real data, suggesting that the model has internalized core periodic and seasonal patterns.

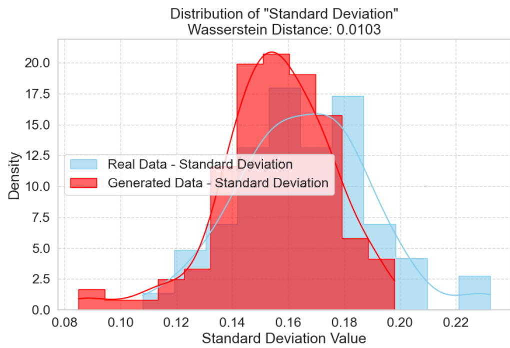
### Discriminative Evaluation

The discriminative classifier achieved an accuracy of 63.74% and an AUC of 0.71 when tasked with distinguishing real from synthetic test samples. While this is not close to the ideal (50% accuracy and 0.5 AUC, indicating indistinguishability), it does indicate that the model has captured meaningful aspects of the real data distribution—though imperfections remain.

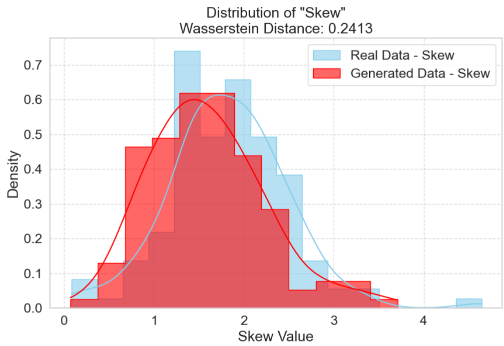
The confusion matrix (Figure 4.3) reveals a tendency to more confidently classify real data as real, while occasionally misclassifying synthetic data as real. This asymmetry suggests the generated samples are not entirely realistic, but contain enough real-like features to challenge the classifier.



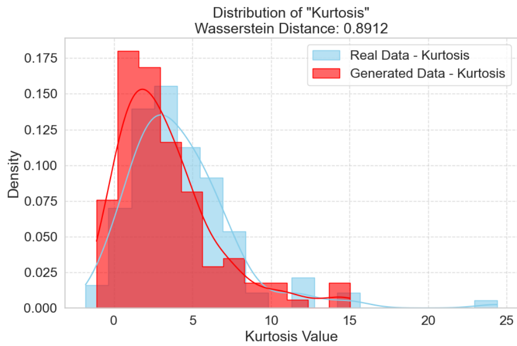

Sample Mean



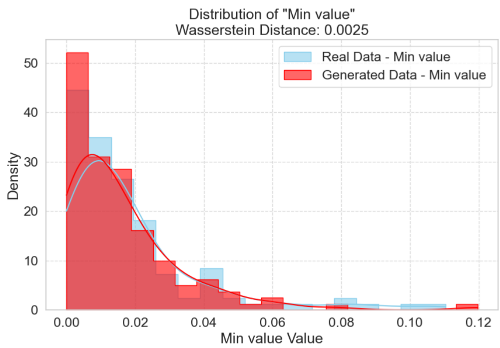

Sample Standard Deviation



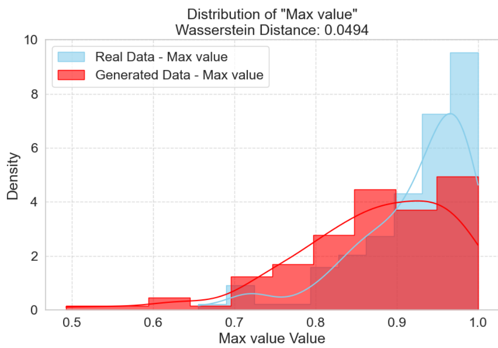

Sample Skew




Sample Kurtosis

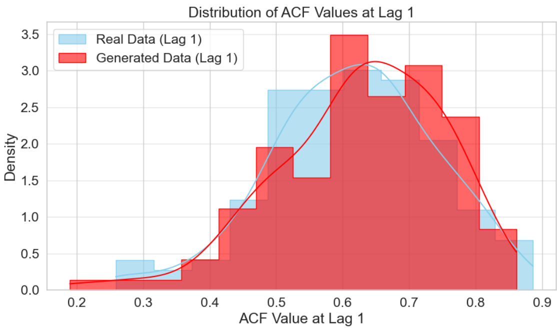



Minimum Value

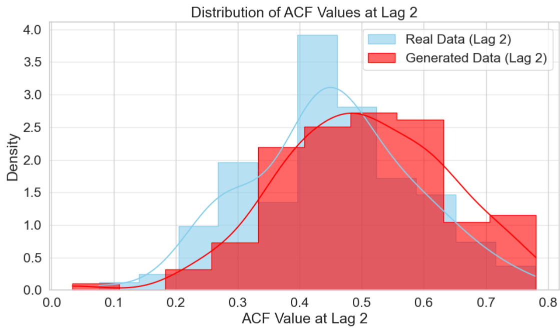



Maximum Value

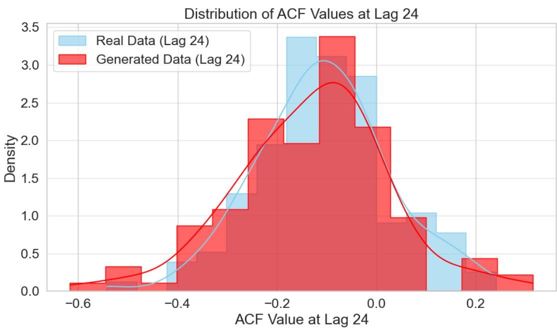
Figure 4.1: Distribution plot of Statistical Features Transfusion



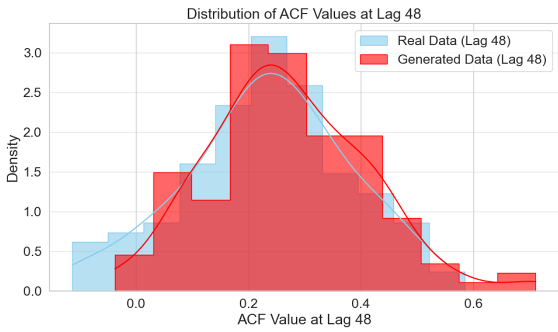

ACF 1



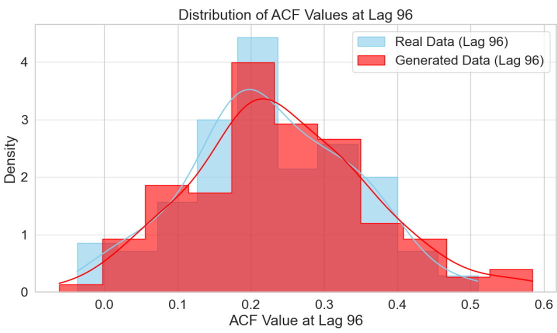

ACF 2



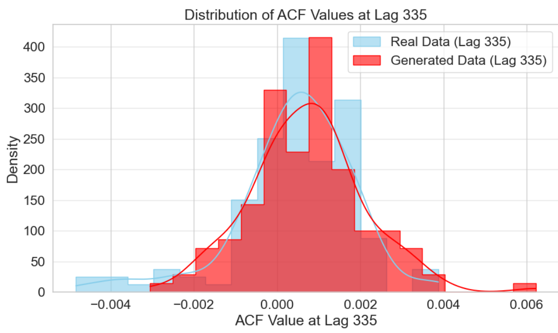

ACF 24




ACF 48




ACF 96




ACF 335

Figure 4.2: Distribution plot of Temporal Features Transfusion

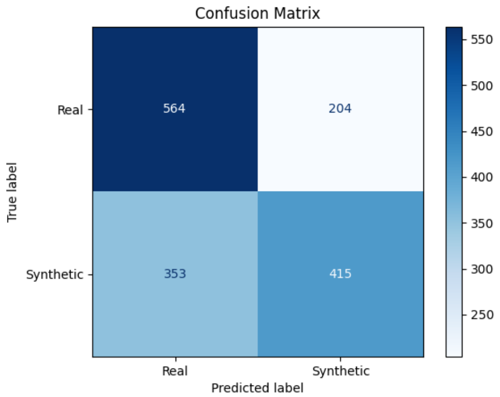



Figure 4.3: Confusion Matrix

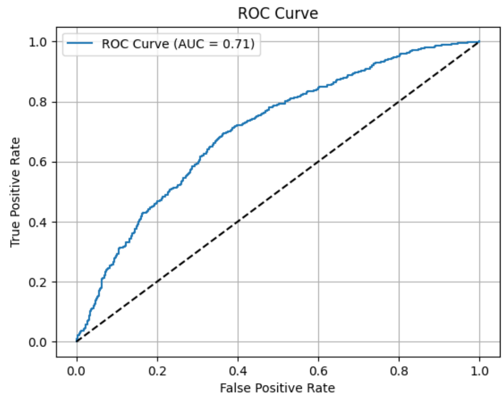



Figure 4.4: ROC

### Train on Synthetic, Test on Real (TSTR)

The forecasting utility of the synthetic data was evaluated using the TSTR paradigm. A forecasting model trained on generated data achieved a Mean Absolute Error (MAE) of 0.0901 on the real test set, compared to 0.0888 for the model trained on real data. This yields a small performance gap of $\Delta\text{MAE} = 1.21 \times 10^{-3}$, indicating that the generated data supports learning predictive models with near-real effectiveness. This is enforced by the diagonal pattern in 4.5, indicating similar performance between the synthetic and real trained forecasting models.

This result suggests that although the generated sequences are not fully indistinguishable from real ones, they retain meaningful temporal and structural information relevant for downstream tasks like forecasting.

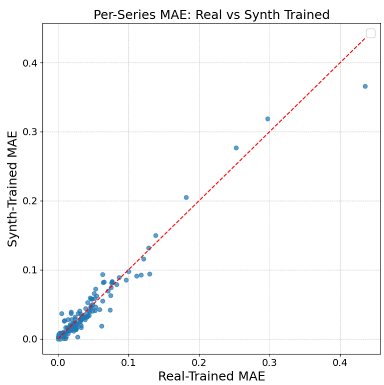



Figure 4.5: Per Series MAE Scatter Plot

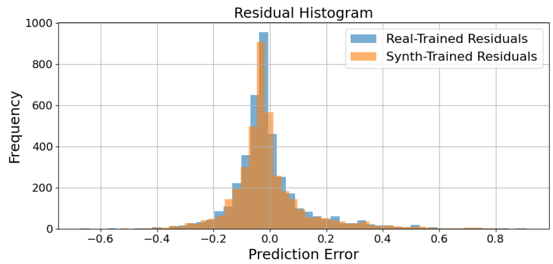



Figure 4.6: Residual Plot

### Direct Data Comparison

To provide a qualitative assessment of the generated data, Figure 4.7 displays a set of randomly sampled sequences from both the generated (top panel) and real (bottom panel) test batches. Each line represents the energy consumption pattern of a single customer over a one-week period (336 timesteps).

At first glance, the plots appear visually dense due to overlapping lines—this reflects the natural variability present across the population rather than noise. Despite the complexity, key patterns can still be discerned:

- Both the real and generated batches exhibit clear daily cyclic patterns, with recurring peaks roughly every 24 timesteps. This indicates that the model has successfully captured the dominant periodic behavior of the data.
- The amplitude and baseline variability are well-matched across the two plots. Peaks in the generated data reach similar heights as those in the real data, and the range of values (from near-zero to around 1.0) is consistent.
- The spread of behavioral diversity is also comparable. In both plots, some sequences show highly active patterns (with frequent high peaks), while others remain relatively flat or less structured—suggesting that the generator has learned to sample from different behavioral modes present in the real dataset.

While the overplotting makes it difficult to distinguish individual sequences, the overall shape and density distribution convey that the TransFusion model reproduces key structural characteristics of the real data.

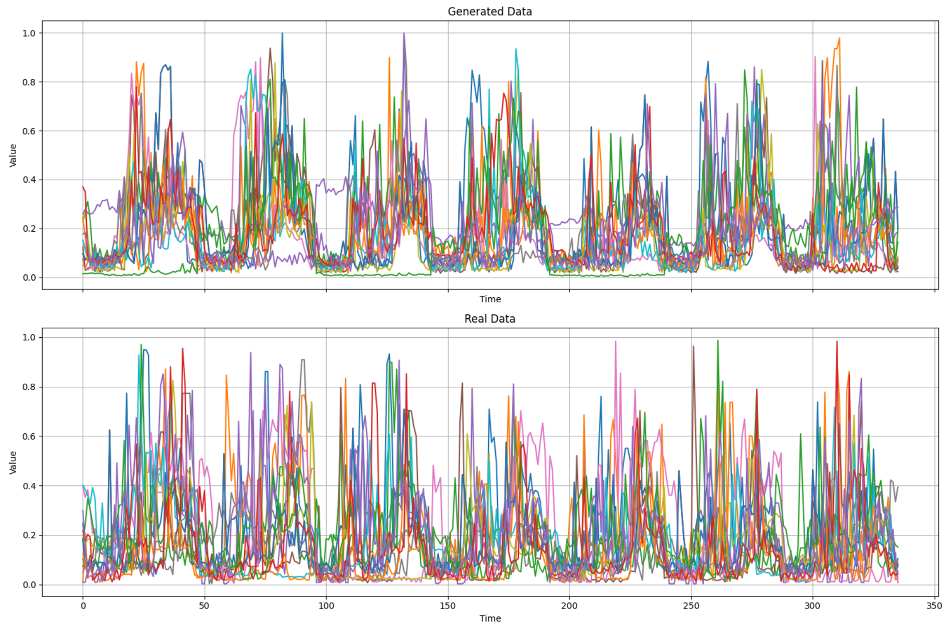



Figure 4.7: Line Plots of Randomly Sampled customers

#### Training Time

The TransFusion model was trained for 5000 epochs using a batch size of 128 and a sequence length of 336. Total training time was 10 hours, 28 minutes, and 3 seconds. While this model achieved strong performance across several evaluation dimensions, its relatively large number of parameters and transformer-based architecture contributed to relatively high training time.

### 4.2.2. Evaluation of the 1D Conditional U-Net Model

The evaluation of the 1D Conditional U-Net model follows the same structure as in Section 4, covering feature distribution analysis, temporal dynamics, discriminative evaluation, and downstream forecasting performance (TSTR).

### Feature Distribution Analysis

Figure 4.8 presents the distributions of statistical features (mean, standard deviation, skewness, kurtosis, minimum, and maximum) computed for both real and generated time series.

The goal of this comparison is to determine how well the model captures global distributional properties across generated sequences. Readers should note:

- A strong match in **minimum, mean, and standard deviation**, with low Wasserstein distances.
- Substantial mismatch in **higher-order moments**—especially kurtosis—indicating the generated data lacks the sharp peaks and long tails found in the real data.
- The plots reflect systematic biases: the generated data tends to exhibit lower mean and standard deviation, and a flatter distribution for kurtosis.

A more detailed analysis of these distributions reveals the following observations:

**Mean.** The distributions of the "Mean" value for both datasets are broadly similar, appearing unimodal and somewhat bell-shaped with a comparable spread. The main distinction is a clear shift in central tendency: the generated data's distribution is centered at a lower mean value (around 0.14) compared to the real data (around 0.18-0.20). The low Wasserstein Distance of 0.0387 indicates good overall similarity despite this systematic shift towards lower mean values in the generated samples.

**Standard Deviation.** The "Standard Deviation" distributions for both real and generated data are unimodal and cover a largely overlapping range. A key difference is the shift in central tendency, with the real data peaking at a higher standard deviation (around 0.18-0.20) and appearing somewhat left-skewed, while the generated data peaks lower (around 0.13-0.14) and seems more symmetric or slightly right-skewed. The Wasserstein Distance of 0.0354, while low, reflects this systematic tendency of the model to generate data with lower standard deviations and a different distributional shape around the peak.

**Skewness.** Both the real and generated data distributions for skewness exhibit a rightward skew and cover a similar range of values. However, they differ notably in their central tendency and dispersion, with the generated data peaking at a higher skew value (around 2.0 vs. 1.5 for real data) and displaying a more spread-out distribution with a heavier right tail. The Wasserstein Distance of 0.4135 reflects this moderate dissimilarity, indicating the generated model tends to produce slightly higher and more variable skewness values.

**Kurtosis.** The distributions of kurtosis for both real and generated data show a strong right skew, with peaks occurring in a similar low kurtosis region (around 2.5-5.0). Significant differences exist in the peak characteristics and spread; the real data has a much sharper, taller peak indicating high concentration, while the generated data's peak is flatter, and its distribution is more dispersed with a noticeably fatter tail, suggesting an overestimation of higher kurtosis values. The substantial Wasserstein Distance of 2.9663 underscores these marked differences in shape and concentration.

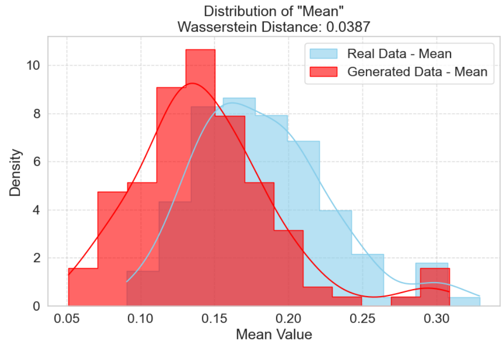
**Minimum Value.** Both real and generated "Min value" distributions are highly concentrated around 0.0 with a strong right skew. The generated data, however, shows a slightly broader peak and a greater spread, particularly extending more into negative values than the real data, which has a taller, more sharply defined peak very close to zero. Despite these subtle differences in peak sharpness and tail behavior, the very low Wasserstein Distance of 0.0130 signifies a high degree of overall similarity between the two.

**Maximum Value.** For the "Max value," both real and generated distributions occupy a similar range (approx. 0.6-1.0) and show the highest density towards the upper end. The primary difference lies in their concentration and skewness: the real data is sharply left-skewed with a prominent peak near 1.0, whereas the generated data is much flatter, more uniformly distributed, and lacks this distinct peak. The Wasserstein Distance of 0.0933 highlights the generated model's difficulty in replicating the strong concentration of maximum values seen in the real data.

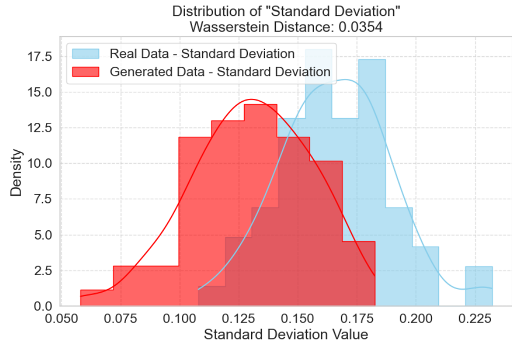
Table 4.2: Wasserstein Distances for Statistical Features (1D Conditional U-Net Model)

| Statistical Feature | Wasserstein Distance |
|---|---|
| Mean | 0.0387 |
| Standard Deviation | 0.0354 |
| Skew | 0.4135 |
| Kurtosis | 2.9663 |
| Min value | 0.0130 |
| Max value | 0.0933 |

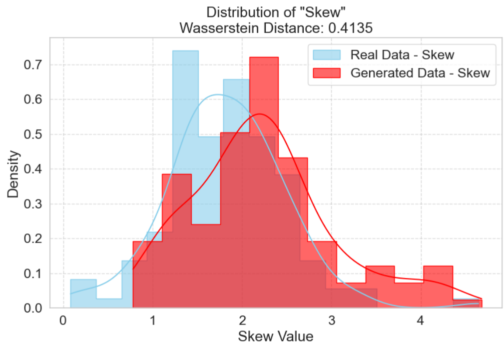
Despite good alignment on basic descriptive metrics, the high distances for skewness and kurtosis suggest that statistical summaries alone are insufficient to capture the full structure of the time series, especially for models with limited capacity.



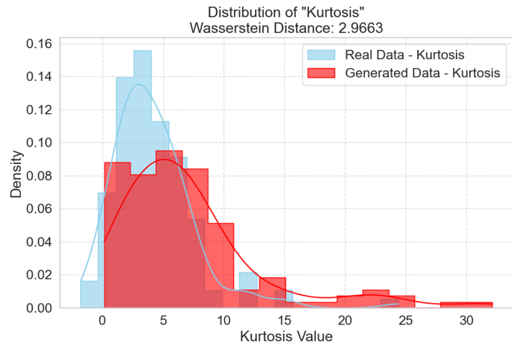

Sample Mean



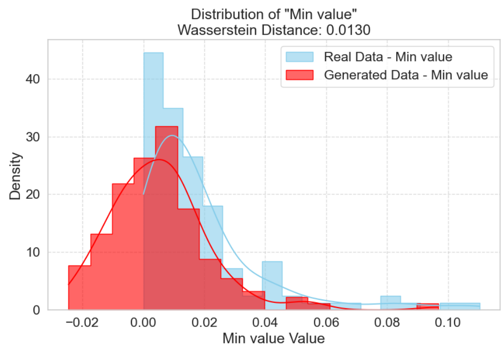

Sample Standard Deviation



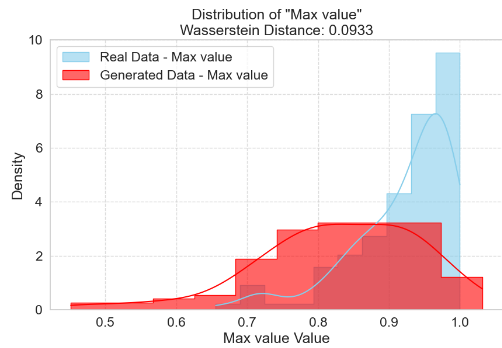

Sample Skewness




Sample Kurtosis




Minimum Value



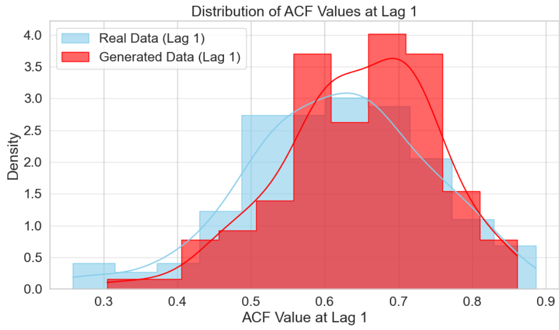

Maximum Value

Figure 4.8: Distribution plot of Statistical Features 1D Cond. Unet

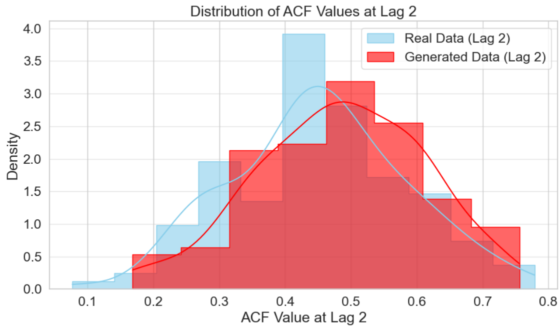
### Temporal Feature Analysis

Figure 4.9 compares the distributions of autocorrelation coefficients at selected lags. This helps assess whether the generated sequences capture the temporal structure of the real data.

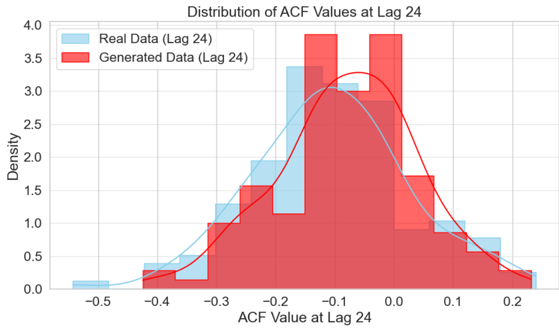
The model performs well on shorter lags (1, 2, 24), indicating that it learns near-term temporal dependencies. However, deviations become noticeable at longer lags (particularly lag 96), where the generated data fails to fully preserve the autocorrelation strength and distributional shape.



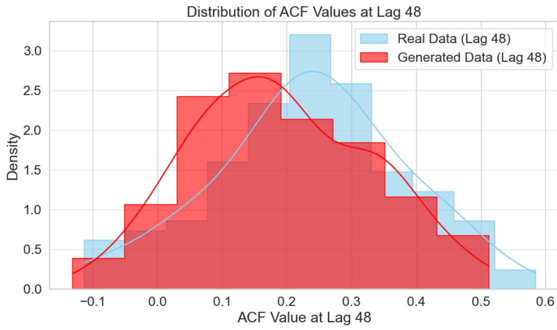

ACF 1



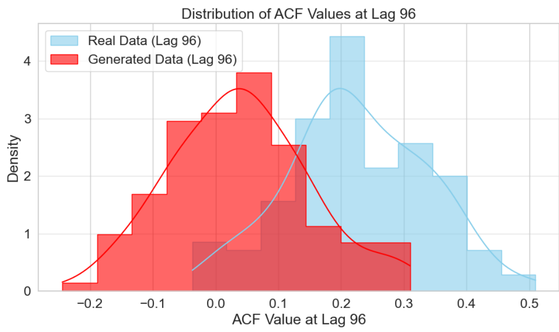

ACF 2



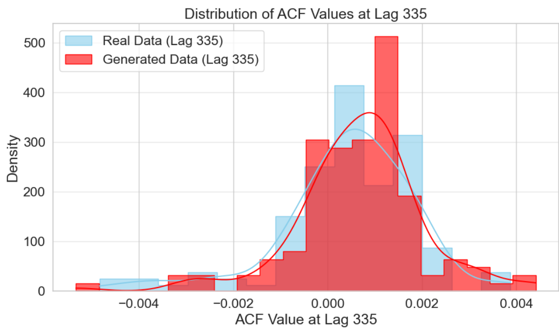

ACF 24




ACF 48




ACF 96




ACF 335

Figure 4.9: Distribution plot of Temporal Features 1D Cond. Unet

## Discriminative Evaluation

Using the classifier described in Section 4, we evaluated the distinguishability of real and generated sequences. The 1D U-Net model performs poorly in this test, with the classifier achieving a near-perfect accuracy of 96.875% and an AUC of 1.0.

As seen in Figure 4.12, the classifier correctly identifies nearly all real and synthetic samples. This indicates that while the generated data may resemble the real data in marginal statistics, it retains identifiable artifacts or structural inconsistencies that the model fails to generalize away.

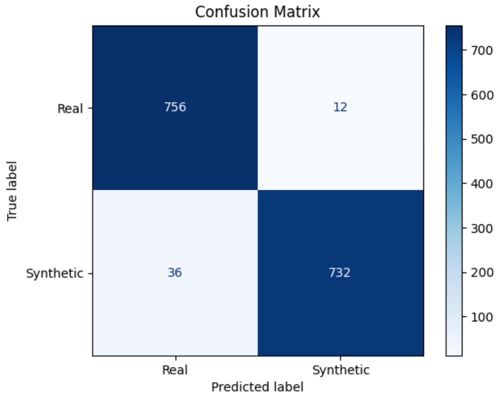



Figure 4.10: Confusion Matrix

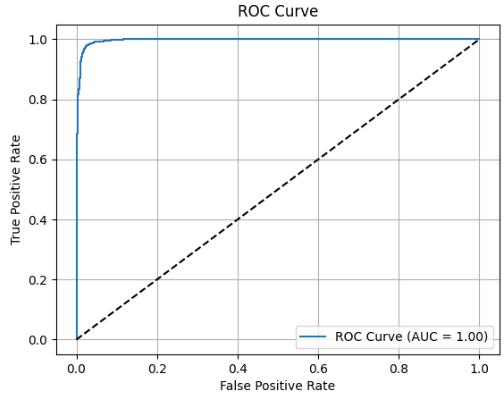



Figure 4.11: ROC Curve

Figure 4.12: Classifier results: high accuracy and perfect AUC indicate poor indistinguishability of generated samples.

## Train on Synthetic, Test on Real (TSTR)

The forecasting utility of the generated data is evaluated via TSTR. The model trained on synthetic data yielded an MAE of 8.7024e-02, while the one trained on real data achieved 8.8849e-02—yielding a gap of $\Delta\text{MAE} = 1.82 \times 10^{-3}$. While a perfect diagonal pattern is not observed in Figure 4.13, a dense cluster of points near the origin indicates that both models achieve low Mean Absolute Error (MAE) on a substantial number of time series.

This suggests the synthetic data supports a reasonable level of learning, though the marginally better performance from real training data implies some information loss.

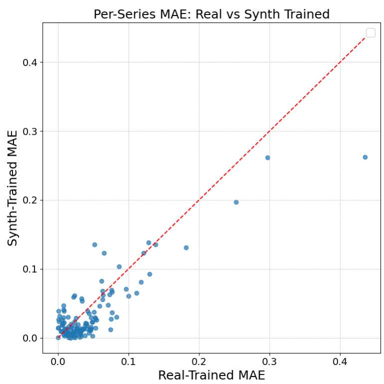



Figure 4.13: Per-series MAE Scatter Plot

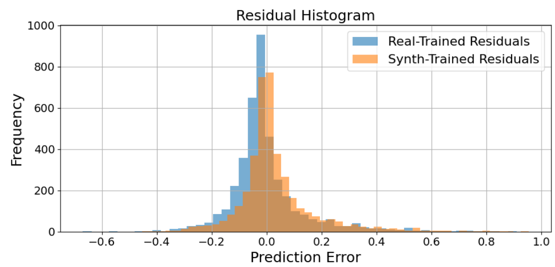



Figure 4.14: Residual Plot

## Direct Data Comparison

To qualitatively assess the realism of generated samples, Figure 4.15 compares randomly selected sequences from real and generated test batches.

Although the overplotting makes individual sequences hard to track, readers should focus on three aspects:

- The generated samples replicate broad seasonal and cyclical trends seen in real data.

- The overall amplitude range and periodicity are visually consistent.
- However, the generated data displays more irregularity and noise, lacking the clear structure and peak separations found in the real data.

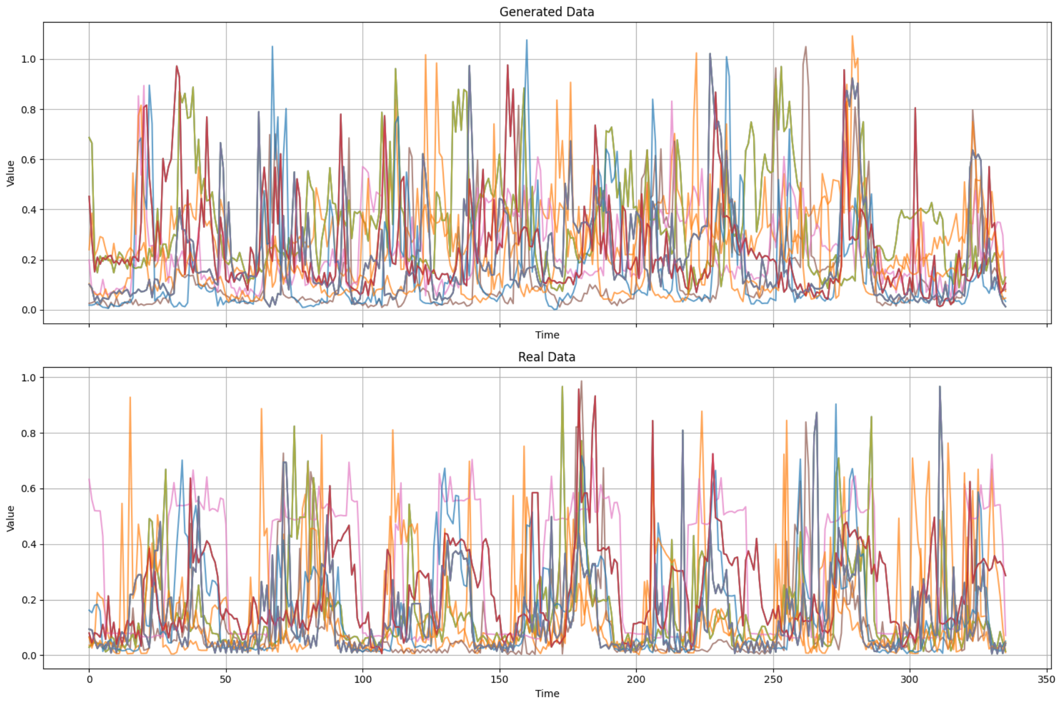



Figure 4.15: Overlay of 50 random sequences from generated (top) and real (bottom) test batches. Readers are encouraged to observe seasonal structure and noise levels.

#### Training Time

The model was trained for 5000 epochs on sequences of length 335. Total training time was 49 minutes and 38 seconds. Despite its efficiency, further improvements to the model architecture may be required to better capture higher-order structure and improve discriminative realism.

### 4.2.3. Evaluation of the Physics-Informed Model

The Physics-Informed model, adapted from Zhang et al. [22], was evaluated using the same metrics as the other architectures. This section presents its performance across statistical, temporal, discriminative, and forecasting dimensions.

#### Feature Distribution Analysis

Figure 4.16 presents the distributions of key statistical features (mean, standard deviation, skewness, kurtosis, minimum, and maximum) for both real and generated sequences.

Overall, the model shows poor statistical fidelity:

- The generated feature distributions diverge significantly in shape and central tendency from the real ones.
- Discrepancies are most pronounced in higher-order moments like kurtosis and skewness, but also present in more fundamental properties such as mean and maximum.
- Only the minimum value shows moderate overlap with real data.

A more detailed analysis of these distributions reveals the following observations:

**Mean.** The Mean distributions are unimodal in both real and generated datasets, but they differ notably in central tendency. The real data peaks around $\mu \approx 0.18$, while the generated data is centered around $\mu \approx 0.07$, resulting in a pronounced left shift. The generated distribution is also slightly more concentrated, suggesting reduced variability in average values. This discrepancy in centering contributes to a moderate Wasserstein Distance of 0.1263, indicating a systematic underestimation of mean load levels by the generative model.

**Standard Deviation.** Both distributions are unimodal with similar shapes, but the real data displays a broader spread and a rightward shift, peaking around $\sigma \approx 0.16$, while the generated data is concentrated near $\sigma \approx 0.09$. The tighter clustering of generated values suggests lower intra-sample variability, possibly indicating a mode collapse tendency. The Wasserstein Distance of 0.0809 quantifies this gap, reflecting a systematic underrepresentation of variability by the model.

**Skewness.** The real data exhibits a wide, right-skewed distribution ranging up to 4.5, with a peak around skew $\approx$ 1.5, while the generated data is more narrowly distributed, peaking sharply around skew $\approx$ 1.0. The generative model underrepresents both the left tail and the more extreme skewness observed in real samples, indicating limited expressivity in replicating asymmetric behavior. The Wasserstein Distance of 0.9148 supports this, pointing to a clear divergence in higher-order moments.

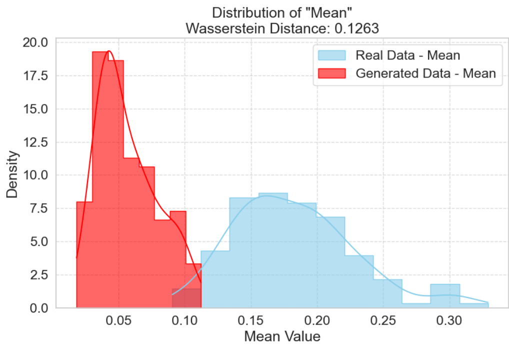
**Kurtosis.** This metric reveals the most pronounced mismatch. The real data is long-tailed with a wide kurtosis range extending up to 25 and a broad peak around kurtosis $\approx$ 3. In contrast, the generated data is tightly concentrated between -1 and 2, failing to replicate the heavy tails and extreme peakedness of the real data. This stark underrepresentation leads to a very high Wasserstein Distance of 4.9106, indicating that the generative model significantly fails to reproduce the distributional sharpness of real samples.

**Minimum Value.** Both distributions are right-skewed and concentrated near zero. However, the real data shows a tighter spread around min $\approx$ 0.0, whereas the generated data includes more negative values, with a broader peak centered around min $\approx -0.03$. Despite these differences in tail behavior, the Wasserstein Distance is relatively low at 0.0469, suggesting decent overall alignment in minimum range characteristics.

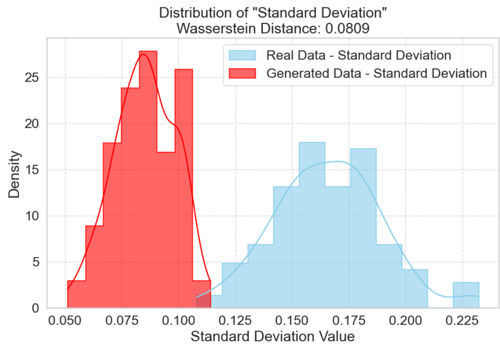
**Maximum Value.** The real data is highly left-skewed with a sharp peak close to max $\approx$ 1.0, reflecting the saturation of values at upper operational limits. Conversely, the generated data shows a roughly symmetric distribution peaking near max $\approx$ 0.28, failing to capture the extreme upper bounds present in the real data. This results in a significant Wasserstein Distance of 0.6591, indicative of a strong underestimation of peak load values by the generative model.

Table 4.3: Wasserstein Distances for Statistical Features (Physics-Informed Model)

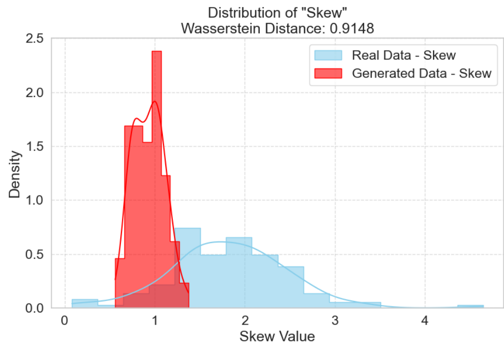
| Statistical Feature | Wasserstein Distance |
|---|---|
| Mean | 0.1263 |
| Standard Deviation | 0.0809 |
| Skewness | 0.9148 |
| Kurtosis | 4.9106 |
| Minimum Value | 0.0469 |
| Maximum Value | 0.6591 |



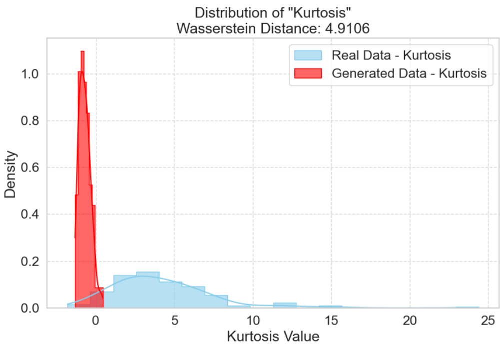

Sample Mean



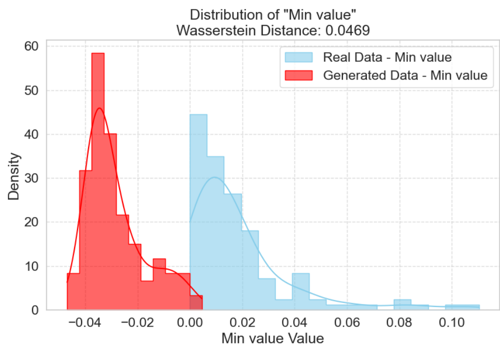

Sample Standard Deviation



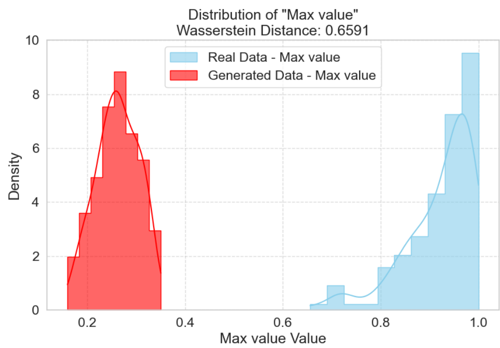

Sample Skewness




Sample Kurtosis




Minimum Value



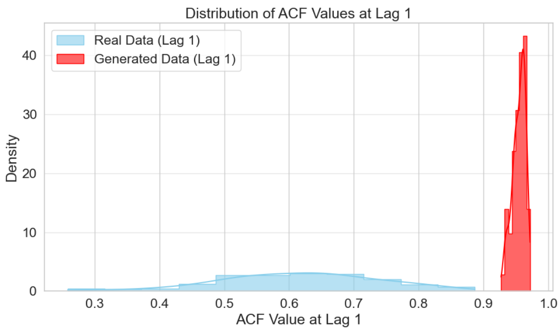

Maximum Value

Figure 4.16: Distribution plot of Statistical Features Physics-Informed

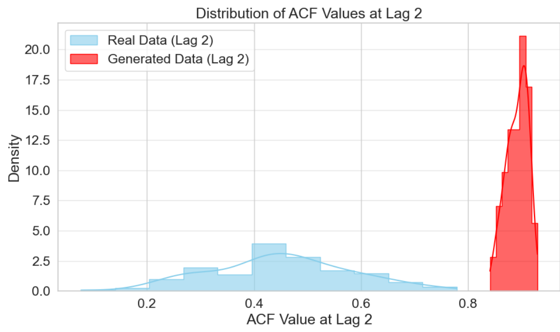
### Temporal Feature Analysis

Figure 4.17 shows the distributions of autocorrelation values at selected lags. While the model captures some degree of short-term temporal correlation (lags 1 and 2), the distributions rapidly diverge at longer lags, particularly lag 96 and 335.

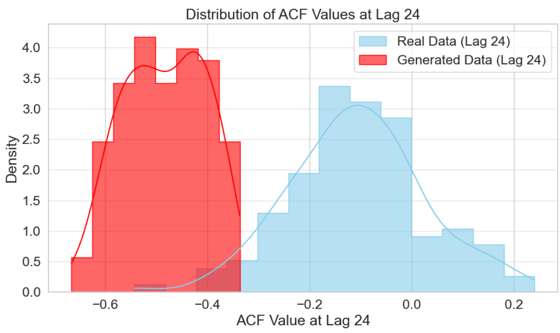
This suggests that although the model encodes periodicity, it fails to replicate long-term dependencies and complex repeating structures that exist in the real data.



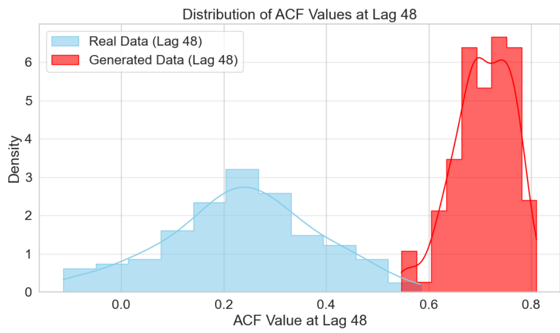

ACF 1



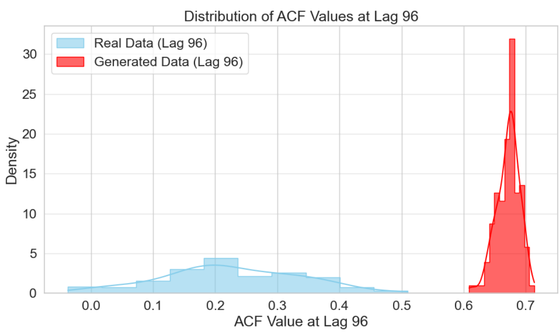

ACF 2



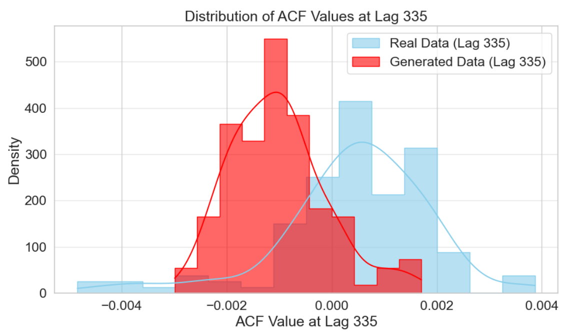

ACF 24




ACF 48




ACF 96




ACF 335

Figure 4.17: Distribution plot of Temporal Features Physics-Informed

### Discriminative Evaluation

Discriminative performance results are shown in Figure 4.20. The classifier was able to distinguish between real and generated data with high confidence—accuracy near 100% and an AUC of 1.0.

This implies that the generated sequences possess distinct artifacts or deviations easily picked up by the classifier, further confirming the statistical mismatch.

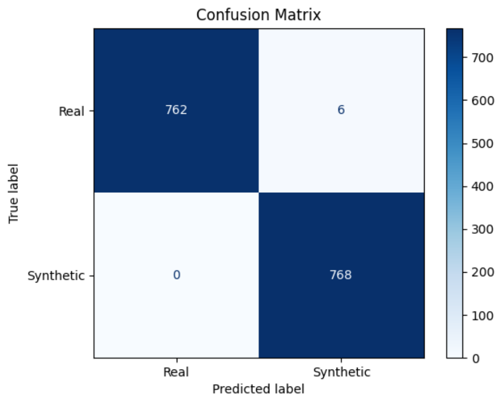



Figure 4.18: Confusion Matrix

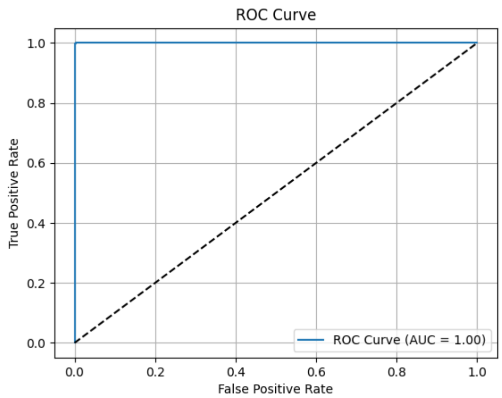



Figure 4.19: ROC Curve

Figure 4.20: Discriminative evaluation results for the Physics-Informed model.

### Train on Synthetic, Test on Real (TSTR)

When evaluated using the TSTR procedure, the model trained on synthetic data achieved a Mean Absolute Error (MAE) of $9.2964 \times 10^{-2}$, while the real-data-trained baseline achieved $8.8849 \times 10^{-2}$—yielding a performance gap of $\Delta\text{MAE} = 4.1155 \times 10^{-3}$. The clustering of points above the diagonal indicates that while the synthetic data captures the overall structure of the real time series, it likely misses finer-grained patterns present in the original data 4.21.

This relatively large error gap highlights the limited downstream utility of the generated samples for this model.

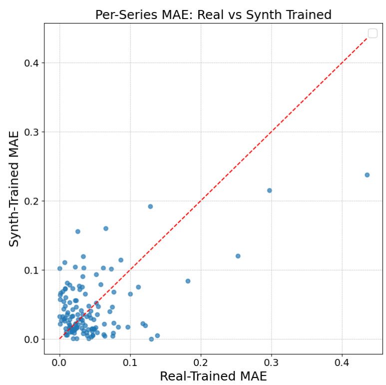



Figure 4.21: Per-series MAE Scatter Plot

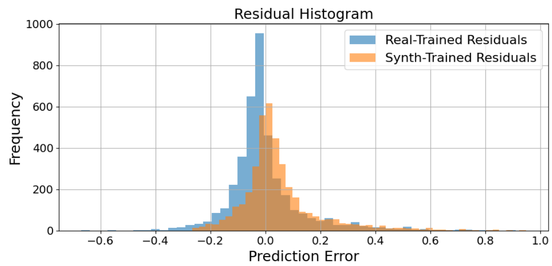



Figure 4.22: Residual Plot

### Direct Data Comparison

Figure 4.23 presents random samples from the real (bottom) and generated (top) batches. Although both exhibit general periodicity and share the same sequence length, there are clear differences:

- The generated signals are compressed in range and amplitude, lacking the full dynamic spread observed in the real data.
- Peaks appear overly smooth and flat, suggesting loss of high-frequency structure.
- Cyclical trends are visible, but more uniform across sequences than in the real data.

This confirms that while the model captures broad seasonality, it fails to replicate finer temporal variation and diverse behavioral modes.

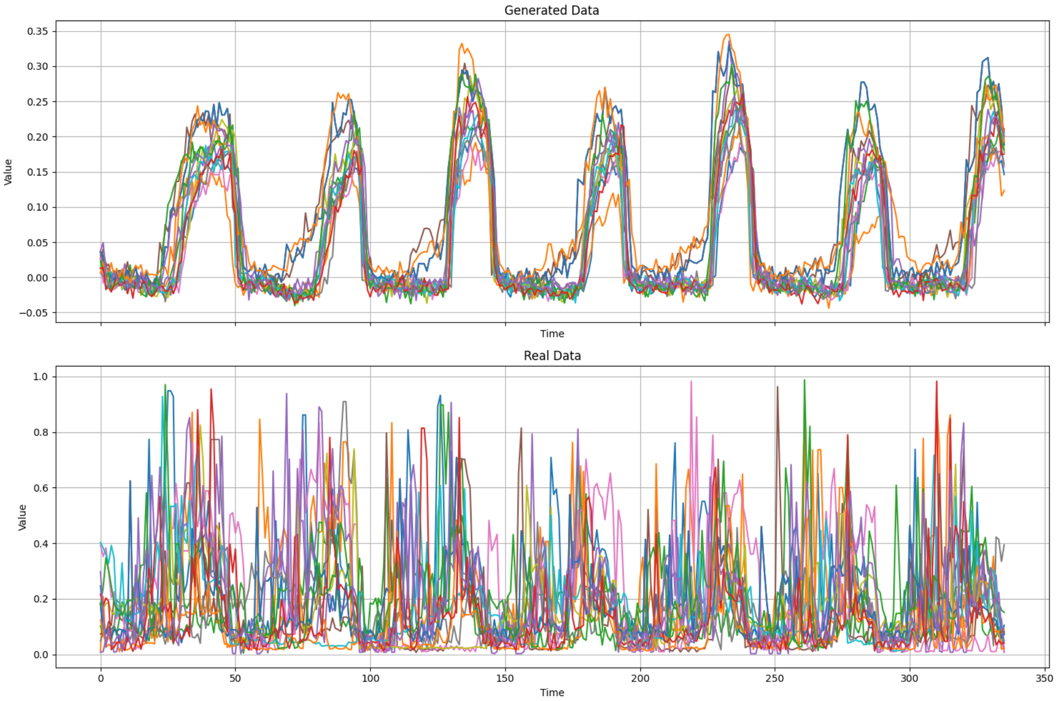



Figure 4.23: Overlay of 50 random sequences from generated (top) and real (bottom) test batches for the Physics-Informed model.

### Training Time

The Physics-Informed model required 13 hours, 10 minutes, and 31 seconds to complete 5000 training epochs using a batch size of 128 and sequence length of 335. Its larger parameter count and hybrid LSTM-attention architecture contributed to longer training duration relative to the other models.

## Model Comparison Summary

To consolidate the results across all three evaluated architectures, Table 4.4 presents a side-by-side comparison of their statistical performance (in terms of Wasserstein distances) and training times. This overview highlights how each model balances fidelity and computational cost.

The 1D Conditional U-Net generally achieves the best performance on basic distributional statistics such as mean, minimum, and skewness, while the TransFusion model excels in standard deviation and maximum value replication. However, both struggle to fully capture higher-order moments like kurtosis. The Physics-Informed model consistently underperforms across nearly all statistical features and also incurs the highest training time, suggesting limited efficiency and effectiveness for this specific generation task.

Table 4.4: Comparison of Wasserstein Distances for Statistical Features and Training Times for TransFusion, 1D Conditional U-Net, and Physics-Informed Models. All models were trained for 5000 epochs with a sequence length of 335.

| Statistical Feature | Model Performance | | |
|---|---|---|---|
| | TransFusion | 1D Cond. U-Net | Physics-Informed |
| *Wasserstein Distance (lower is better)* | | | |
| Mean | 0.0136 | **0.0132** | 0.1284 |
| Standard Deviation | **0.0027** | 0.0219 | 0.0832 |
| Skewness | 0.1962 | **0.1632** | 0.9027 |
| Kurtosis | **1.2110** | 1.3457 | 4.8779 |
| Minimum Value | 0.0094 | **0.0064** | 0.0472 |
| Maximum Value | **0.0291** | 0.0501 | 0.6663 |
| $\Delta MAE$ | **1.2093e-03** | 1.8244e-03 | 4.1155e-03 |
| *Computational Cost* | | | |
| Training Time | 10h 25m 36s | 49m 38s | 13h 10m 31s |

## RQ1: Comparative Hypothesis Testing of Promising Models

To address the first research question, two hypothesis tests are conducted:

1. Comparing forecasting performance between models trained on real versus synthetic data.
2. Comparing forecasting performance between two models trained on synthetic data from *TransFusion* and *1D Conditional U-Net*.

Since the Shapiro-Wilk test indicates that the data is not normally distributed, the non-parametric Mann-Whitney U test is employed. For both tests, the null hypothesis assumes no difference between the groups being compared. A significance level of $\alpha = 0.05$ is used throughout. The forecasting models are trained and evaluated using 5-fold cross-validation with 6 repetitions.

### 4.2.4. Real vs Synthetic Data Comparison

Table 4.5 summarizes the performance comparison between models trained on real versus synthetic data for both the TransFusion and 1D Conditional U-Net models.

Table 4.5: Real vs Synthetic Data Performance Comparison

| Model | Metric | Real Mean (Std) | Synthetic Mean (Std) | p-value | Cohen's d |
|---|---|---|---|---|---|
| TransFusion | MAE | 5.22e-02 (0.0021) | 4.95e-02 (0.0024) | 3.34e-03 | 1.20 |
| TransFusion | RMSE | 8.06e-02 (0.0083) | 7.43e-02 (0.0058) | 1.84e-01 | 0.89 |
| 1D Cond U-Net | MAE | 5.22e-02 (0.0021) | 1.42e-02 (0.0006) | 2.42e-11 | 24.4 |
| 1D Cond U-Net | RMSE | 8.06e-02 (0.0083) | 1.87e-02 (0.0011) | 2.42e-11 | 10.5 |

For the TransFusion model, the MAE shows a statistically significant difference between real and synthetic data (p = 3.34e-03), while RMSE does not (p = 1.84e-01). For the 1D Conditional U-Net, both MAE and RMSE differences are highly significant (p = 2.42e-11 for both), with extremely large effect sizes. As the forecasting horizon increases, the statistical significance tends to strengthen, indicated by decreasing p-values at longer horizons.

#### Model Comparison: TransFusion vs 1D Conditional U-Net

Table 4.6 reports the statistical comparison between the TransFusion and 1D Conditional U-Net models trained on synthetic data.

Table 4.6: Performance Comparison Between TransFusion and 1D Conditional U-Net (Synthetic Data)

| Metric | TransFusion Mean (Std) | 1D Cond U-Net Mean (Std) | p-value | Cohen's d |
|---|---|---|---|---|
| MAE | 9.07e-02 (0.0016) | 4.33e-02 (0.0026) | 2.42e-11 | 21.5 |
| RMSE | 1.35e-01 (0.0018) | 7.61e-02 (0.0053) | 2.42e-11 | 16.4 |

The results show significant differences in forecasting performance between the two models across both MAE and RMSE metrics, with large effect sizes. These findings hold consistently across different forecasting horizons.

## 4.3. RQ2: How do different embedding mechanisms impact the performance of diffusion models in generating or augmenting customer energy consumption profiles?

This section presents the experimental results for the second research question, focusing on two primary investigations: firstly, the comparative performance of different conditioning embedding mechanisms relative to the default Multi-Layer Perceptron (MLP) embedding, and secondly, the influence of employing cross-attention for integrating conditioning data versus the standard concatenation method.

### 4.3.1. Comparative Evaluation of Embedding Mechanisms

This section presents a detailed evaluation of six distinct time series embedding approaches for the conditional diffusion model. The assessment is based on three primary criteria: firstly, the fidelity in replicating statistical features of the real data, quantified by Wasserstein distances ($W$), secondly, the realism of generated samples as determined by a discriminative classifier, measured by accuracy and ROC AUC, and thirdly the performance on the TSTR evaluation. Lower Wasserstein distances indicate better statistical similarity. For the classifier, an accuracy approaching 50% and an ROC AUC nearing 0.5 suggest that generated samples are difficult to distinguish from real data, implying higher realism.

The quantitative results for statistical feature fidelity and TSTR evaluation are summarized in Tables 4.7 and 4.8, while classifier performance metrics and training details are presented in Table 4.9.

It is important to note a difference in training duration for one configuration: due to substantial computational requirements, the model variant employing the Transformer embedding mechanism was trained for only 500 epochs. In contrast, models utilizing other embedding mechanisms completed the standard 5000 training epochs.

Table 4.7: Wasserstein Distances for Statistical Features by Embedding Mechanism (Lower $W$ is Better; lowest per feature in **bold**)

| Embedding | Mean($W$) | SD($W$) | Skew($W$) | Kurt($W$) | Min($W$) | Max($W$) |
|---|---|---|---|---|---|---|
| Default (MLP) | 0.0070 | 0.0103 | 0.2413 | 0.8912 | **0.0025** | 0.0494 |
| FFT (im) | 0.0191 | 0.0035 | 0.1763 | 0.5502 | 0.0044 | 0.0281 |
| FFT (mag) | **0.0050** | 0.0035 | **0.1027** | 0.8222 | 0.0043 | 0.0287 |
| STFT (MLP) | 0.0804 | **0.0021** | 0.6404 | 2.7212 | 0.0397 | 0.0387 |
| STFT (CNN) | 0.0290 | 0.0026 | 0.2394 | 0.7303 | 0.0055 | 0.0306 |
| Transformer | 0.0146 | 0.0047 | 0.1065 | **0.3681** | 0.0054 | **0.0131** |

Table 4.8: Forecasting Performance (Δ MAE) by Embedding Mechanism in TSTR Evaluation (Lower is Better)

| Embedding | Δ MAE |
|---|---|
| Default (MLP) | 1.2093e-03 |
| FFT (im) | 3.0148e-03 |
| FFT (mag) | 1.3781e-03 |
| STFT (MLP) | 1.0089e-01 |
| STFT (CNN) | 6.3624e-04 |
| Transformer | **2.4775e-04** |

Table 4.9: Classifier Performance and Training Details by Embedding Mechanism (Classifier Acc. near 50% and AUC near 0.5 indicate higher realism)

| Embedding | Accuracy(%) | AUC | Epochs | Training Time |
|---|---|---|---|---|
| Default (MLP) | 63.74 | 0.71 | 5000.00 | 10h 28m 3s |
| FFT (im) | 57.16 | **0.61** | 5000.00 | 10h 23m 54s |
| FFT (mag) | **56.97** | 0.66 | 5000.00 | 10h 47m 14s |
| STFT (mlp) | 83.66 | 0.93 | 5000.00 | 11h 03m 50s |
| STFT (cnn) | 63.15 | 0.69 | 5000.00 | 10h 33m 42s |
| Transformer | 61.78 | 0.66 | 500.00 | 1d 2h 37m 1s |

**Default (MLP) Embedding.** The baseline MLP embedding showed strong performance in replicating the **minimum value** distribution ($W$ = **0.0025**, Table 4.7) and was competitive for the **mean** ($W$ = 0.0070). However, it exhibited moderate to higher Wasserstein distances for standard deviation ($W$ = 0.0103), skewness ($W$ = 0.2413), kurtosis ($W$ = 0.8912), and maximum values ($W$ = 0.0494). In the discriminative evaluation (Table 4.9), this embedding led to a classifier accuracy of 63.74% and an ROC AUC of 0.71. The confusion matrix revealed that 73.44% of real samples were correctly classified, while a lower proportion of synthetic samples (54.04%) were correctly identified as synthetic, suggesting some difficulty for the classifier in distinguishing synthetic samples from real ones under certain conditions.

**FFT (imaginary part) Embedding.** The FFT (im) embedding performed well in capturing **kurtosis** ($W = 0.5502$, Table 4.7) and **minimum values** ($W = 0.0044$). Its performance was moderate for other statistical features: mean ($W = 0.0191$), standard deviation ($W = 0.0035$), skewness ($W = 0.1763$), and maximum values ($W = 0.0281$). Regarding distinguishability (Table 4.9), this embedding achieved a classifier accuracy of 57.16%, the closest to the 50% target among the evaluated methods, with an ROC AUC of 0.61. This suggests a higher degree of realism. The confusion matrix indicated that while 72.14% of generated samples were correctly identified as synthetic, only 42.19% of real samples were correctly classified as real, implying that real samples were more often mistaken for synthetic ones. Evaluating the embedding mechanism on the TSTR evaluation we see that it does significantly worse than the default MLP embedding.

**FFT (magnitude) Embedding.** This embedding demonstrated strong performance for several fundamental statistical properties (Table 4.7), achieving the best Wasserstein distances for **mean** ($W = \mathbf{0.0050}$) and **skewness** ($W = \mathbf{0.1027}$), and a very competitive result for **minimum values** ($W = 0.0043$). It also performed well for standard deviation ($W = 0.0035$) and maximum values ($W = 0.0287$), although its kurtosis distance ($W = 0.8222$) was moderate. The classifier achieved 56.97% accuracy and a 0.66 ROC AUC with FFT (mag) embedding (Table 4.9), also indicating good realism. The confusion matrix showed a high correct classification rate for real samples (86.46%), but a low rate for synthetic samples (27.47%), suggesting synthetic samples were often mistaken for real ones. This embedding method scores slightly worse than the default MLP on the TSTR evaluation.

**STFT (MLP) Embedding.** The STFT embedding excelled in matching the **standard deviation** distribution ($W = \mathbf{0.0021}$, Table 4.7). However, it significantly underperformed across all other statistical metrics: mean ($W = 0.0804$), skewness ($W = 0.6404$), kurtosis ($W = 2.7212$), minimum ($W = 0.0397$), and maximum values ($W = 0.0387$). This poorer statistical fidelity was reflected in the classifier performance (Table 4.9), which recorded an accuracy of 83.66% and an ROC AUC of 0.93. These high scores indicate that samples generated using STFT embedding were easily distinguished from real data. This model also scored the worst in terms of $\Delta MAE$.

**STFT (CNN) Embedding.** Replacing the MLP with a CNN for spectrogram processing, the STFT (cnn) embedding yielded a dramatic improvement over its MLP-based counterpart. It was highly competitive in replicating the **standard deviation** (W=0.0026), second only to the STFT (mlp), and showed solid, mid-range performance across all other statistical features. Crucially, its classifier performance (63.15% accuracy, 0.69 AUC) was vastly superior to the STFT (mlp) model and nearly identical to the Default (MLP) baseline. This demonstrates that a convolutional architecture is significantly more adept at extracting meaningful features from spectrograms for this generative task. Despite the MLP version being the worst performer on the TSTR in terms of $\Delta MAE$, the CNN version scores a close second best.

**Transformer Encoder Embedding.** Despite its limited training duration, the Transformer embedding showed excellent performance in reproducing **kurtosis** ($W = \mathbf{0.3681}$, Table 4.7) and **maximum value** distributions ($W = \mathbf{0.0131}$). It was also competitive for skewness ($W = 0.1065$) and minimum values ($W = 0.0054$). Its distances were moderate for the mean ($W = 0.0146$) and higher for standard deviation ($W = 0.0047$). In the discriminative task (Table 4.9), it achieved an accuracy of 61.78% with an ROC AUC of 0.66. The confusion matrix indicated correct classification rates of 67.80% for real data and 55.60% for generated data. The transformer embedding excelled on the TSTR evaluation scoring the best of all models.

### 4.3.2. RQ2.1: Comparative Hypothesis Testing of Promising Models

This hypothesis test compares the performance of the STFT-CNN embedding against the Transformer-based embedding. As in previous tests, the Mann-Whitney U test is employed due to the non-normal distribution of the data, with a significance level of $\alpha = 0.05$.

The results reveal statistically significant differences between the two embedding methods for both MAE and RMSE metrics. The MAE comparison yields a p-value of 2.42e-11 and a Cohen's $d$ of -11.4, while the RMSE comparison produces the same p-value and a Cohen's $d$ of -8.98.

Table 4.10: Performance Comparison: STFT-CNN vs Transformer Embedding

| Metric | Embedding | Mean | Std | Cohen's d / p-value |
|---|---|---|---|---|
| MAE | STFT-CNN | 4.83e-02 | 0.0015 | -11.4 / 2.42e-11 |
| | Transformer | 7.29e-02 | 0.0027 | |
| RMSE | STFT-CNN | 7.66e-02 | 0.0028 | -8.98 / 2.42e-11 |
| | Transformer | 1.15e-01 | 0.0053 | |

### 4.3.3. Impact of Cross-Attention for Conditioning Integration

This section evaluates the impact of using a cross-attention mechanism for integrating conditioning data with the primary meter data, comparing its performance against the default model which employs standard concatenation. The evaluation considers both statistical feature fidelity and classifier-based distinguishability.

Table 4.11: Wasserstein Distances for Statistical Features: Default (Concatenation) vs. Cross-Attention Model (Lower $W$ is Better)

| Statistical Feature | Wasserstein Distance Default | Wasserstein Distance Cross-Attention |
|---|---|---|
| Mean Value | **0.0070** | 0.0168 |
| Standard Deviation | 0.0103 | **0.0057** |
| Skewness | 0.2413 | **0.1855** |
| Kurtosis | 0.8912 | **0.7434** |
| Minimum Value | **0.0025** | 0.0043 |
| Maximum Value | 0.0494 | **0.0126** |

**Statistical Feature Comparison.** Table 4.11 presents the Wasserstein distances for key statistical features. The cross-attention model demonstrates improved performance (lower $W$ distances) over the default concatenation model for several features: **standard deviation** (0.0057 vs. 0.0103), **skewness** (0.1855 vs. 0.2413), **kurtosis** (0.7434 vs. 0.8912), and notably for the **maximum value** (0.0126 vs. 0.0494). Conversely, the default model maintained better fidelity for the **mean value** (0.0070 vs. 0.0168) and the **minimum value** (**0.0025** vs. 0.0043). Regarding temporal characteristics not detailed in the table, the distributions of the Autocorrelation Function (ACF) values at various lags appeared largely similar for both the default and cross-attention models.

**Train Synthetic Test Real.** The model was further evaluated using the Train on Synthetic, Test on Real (TSTR) framework. The results indicate that the model performed reasonably well under this evaluation. The observed $\Delta$MAE between the predictors trained on the base model and the cross-attention model was $1.8031 \times 10^{-3}$. Figures 4.24 and 4.25 illustrate the MAE differences across individual time series within the batch, as well as the corresponding residual distributions.

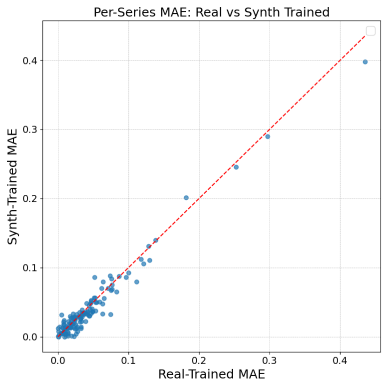



Figure 4.24: Per Series MAE Scatter Plot

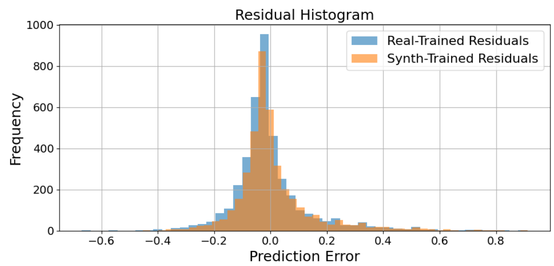



Figure 4.25: Residual Plot

**Classifier Performance.** The performance of the discriminative classifier, tasked with distinguishing real samples from those generated by the models, is illustrated by the confusion matrices in Figure 4.28 and summarized by accuracy and ROC AUC scores.

The cross-attention model resulted in a classifier accuracy of 65.10% and an ROC AUC of 0.73. These scores are marginally higher than those from the default model (which achieved 63.74% accuracy and 0.71 ROC AUC, as noted in Table 4.9). This indicates that the classifier was slightly more effective at distinguishing samples generated by the cross-attention model from real data, suggesting that, in this specific aspect of classifier-based evaluation, the cross-attention mechanism did not enhance the realism of the generated samples compared to the default concatenation method.

Analyzing the confusion matrix for the cross-attention model (Figure 4.27), 76.12% of true real samples were correctly classified, while 54.04% of true synthetic samples were correctly identified. This compares to the default model's performance where 73.44% of real samples and 54.04% of synthetic samples were correctly classified (Figure 4.26). This suggests that while both models achieved similar success in identifying synthetic samples, the cross-attention mechanism led to a marginal increase in the correct classification of real samples by the discriminator.

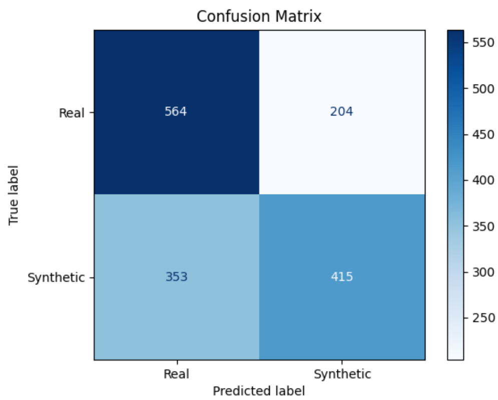



Figure 4.26: Confusion Matrix Default Model

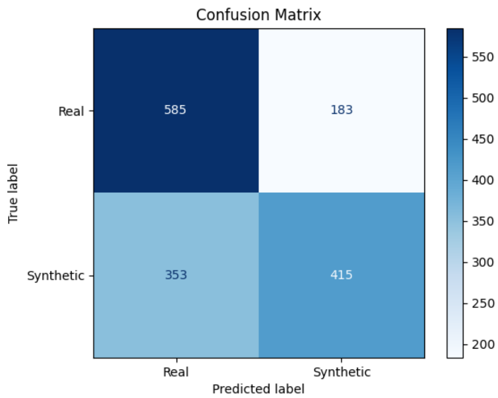



Figure 4.27: Confusion Matrix Cross-Attention Model

Figure 4.28: Comparison of Confusion Matrices for Default (Concatenation) and Cross-Attention Models.

### 4.3.4. RQ2.2: Comparative Hypothesis Testing of Promising Models

To address this part of the research question, we compare the forecasting performance of the standard concatenation-based TransFusion model with its cross-attention variant, referred to as *TransFusionX*. The non-parametric Mann-Whitney U test is again employed, as the data does not satisfy normality assumptions. The null hypothesis assumes no difference between the models, with a significance level of $\alpha = 0.05$.

The results indicate no statistically significant differences between the two models for the MAE. The MAE comparison yields a p-value of 8.08e-10 and a Cohen's $d$ of 1.82. The Hypothesis test does indicate a significant difference between the two models for the RMSE metric. The RMSE comparison also produces a p-value of 8.08e-10 and a Cohen's $d$ of 1.77.

Table 4.12: Performance Comparison: TransFusion vs TransFusionX

| Metric | Model | Mean | Std | Cohen's d / p-value |
|---|---|---|---|---|
| MAE | TransFusion | 5.90e-02 | 0.0022 | -3.15e-02 / 7.95e-01 |
| | TransFusionX | 5.91e-02 | 0.0022 | |
| RMSE | TransFusion | 8.90e-02 | 0.0064 | 1.01e+00 / 5.22e-04 |
| | TransFusionX | 8.38e-02 | 0.0035 | |

## 4.4. RQ3: How do different embedding variables impact the performance of diffusion models in generating or augmenting customer energy consumption profiles?

This section presents the experimental results pertaining to the third research question, which investigates the impact of different conditioning data types on the performance of the Denoising Diffusion Probabilistic Model (DDPM). While previous sections utilized features engineered from the date of the meter reading (referred to as 'Default (Time features)'), this analysis explores the efficacy of incorporating weather-derived features (detailed in 3.1.2) and statistical features extracted from the input time series.

**Impact of All Weather Features** The set of weather variables incorporated as conditioning data included: temperature (actual, feel, minimum, maximum), dew point, atmospheric pressure, humidity, wind speed and direction, cloud coverage, and weather type (e.g., clear, clouds, rain, snow).

As indicated in Table 4.13, the model conditioned solely on default time features generally exhibited lower Wasserstein distances across several statistical moments compared to the model conditioned on time and weather features. Notably, the Wasserstein distances for skewness ($W = 0.3117$ vs. $W = 0.2413$) and kurtosis ($W = 1.1112$ vs. $W = 0.8912$) were higher for the weather feature-conditioned model, signifying a poorer match with the statistical properties of the real data in these aspects. Conversely, the weather-conditioned model showed a lower Wasserstein distance for the standard deviation ($W = 0.0038$ vs. $W = 0.0103$) and maximum value ($W = 0.0331$ vs. $W = 0.0494$).

The confusion matrices presented in Figure 4.31 illustrate a shift in classifier performance. The default model demonstrated higher efficacy in classifying real samples, whereas the classifier struggled with synthetic samples. Conversely, when weather features were incorporated, the classifier found synthetic samples easier to identify but experienced greater difficulty in correctly classifying real samples. Regarding discernibility metrics, the introduction of weather features led to a slight decrease in the Area Under the ROC Curve (AUC) from 71% to 70% and a decrease in accuracy from 63.74% to 61.98% when distinguishing between real and synthetic samples.

Including the weather features did not lead to any improvement in the TSTR evaluation performance. In fact, the performance slightly declined compared to the baseline model. The model with weather features yielded a ΔMAE of $1.2108 \times 10^{-3}$, whereas the baseline model achieved a ΔMAE of $1.2093 \times 10^{-3}$.

Table 4.13: Wasserstein Distances for Default (Time Features) vs. Time + Weather Features Models (Lower $W$ is Better)

| **Embedding** | **Mean**($W$) | **SD**($W$) | **Skew**($W$) | **Kurt**($W$) | **Min**($W$) | **Max**($W$) |
|---|---|---|---|---|---|---|
| Default (Time features) | **0.0070** | 0.0103 | **0.2413** | **0.8912** | **0.0025** | 0.0494 |
| Time + weather features | 0.0340 | **0.0038** | 0.3117 | 1.1112 | 0.0059 | **0.0331** |

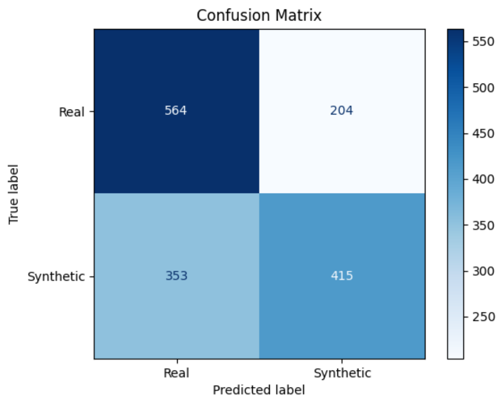



Figure 4.29: Confusion Matrix Default Model

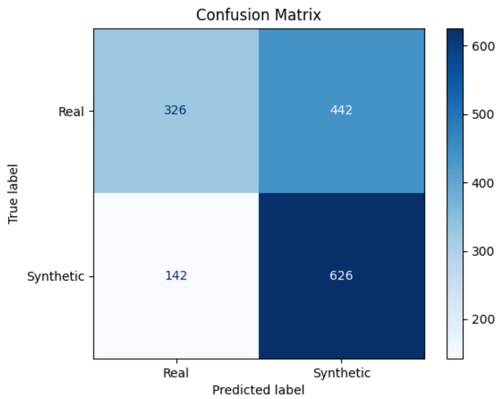



Figure 4.30: Confusion Matrix Time + Weather Features Model

Figure 4.31: Comparison of Confusion Matrices for Default Model and Time + Weather Features Model.

**Impact of Different weather variables** Not all weather features influence energy consumption to the same extent, as demonstrated by [14]. Therefore, we analyze the differences in energy consumption prediction performance when using: (i) all available weather features, (ii) only humidity-related features, (iii) only temperature-related features, (iv) only cloud-related features, and (v) weather and humidity-related features since these appear to be the features with the greatest impact.

Based on Table 4.14, the temperature features exhibit the best overall performance on the statistical metric evaluation. Specifically, they achieve the lowest Wasserstein distance for the mean, skew, and kurtosis, as well as a three-way tie for the minimum value. They also rank second in standard deviation, with only a 1-point difference from the top model. Interestingly, models that use more selective feature sets consistently outperform the model that includes all weather features. This suggests that a focused selection of variables can be more effective than a comprehensive approach, potentially because including less relevant features introduces noise or redundancy.

Evaluating the impact of using only temperature features combined with the default time features as conditioning data, we observe that despite improved statistical metrics, the classifier's performance in distinguishing real from synthetic data increased slightly compared to the baseline: Accuracy increased from 61.98% to 64%, and ROC improved from

Table 4.14: Wasserstein Distances for Time + Weather Feature Variants (Lower $W$ is Better; lowest per column in **bold**)

| Feature Group | Mean($W$) | SD($W$) | Skew($W$) | Kurt($W$) | Min($W$) | Max($W$) |
|---|---|---|---|---|---|---|
| Weather (All) | 0.0340 | 0.0038 | 0.3117 | 1.1112 | 0.0059 | 0.0331 |
| Temperature Only | **0.0319** | 0.0031 | **0.2546** | **0.7739** | **0.0058** | 0.0349 |
| Humidity Only | 0.0328 | 0.0033 | 0.2721 | 0.8670 | **0.0058** | 0.0331 |
| Cloud Only | 0.0330 | **0.0030** | 0.2825 | 0.9003 | **0.0058** | **0.0312** |
| Temp. + Humid. | 0.0333 | 0.0033 | 0.2759 | 0.8577 | 0.0059 | 0.0326 |

Table 4.15: Forecasting Error (Δ MAE) for Models with Weather-Based Feature Groups (Lower is Better)

| Feature Group | Δ MAE |
|---|---|
| Weather (All) | 1.2108e-03 |
| Temperature Only | 1.2571e-03 |
| Humidity Only | 1.3456e-03 |
| Cloud Only | **1.2077e-03** |
| Temp. + Humid. | 1.3686e-03 |

70% to 71%. Importantly, the confusion matrix reveals that the classifier correctly identified fewer synthetic samples (decreasing from 81% to 73%), while correctly identifying real samples improved from 42.45% to 55%. The TSTR evaluation 4.15 showed that the ΔMAE got worse from 1.2108e-03 to 1.2571e-03.

The humidity features yielded slightly lower classifier performance in terms of Accuracy and ROC, achieving 61.85% and 68%, respectively. Notably, the confusion matrix of this model resembles more closely that of the time feature-only matrix rather than the time and weather combined matrix. Here, the classifier struggled significantly more to identify synthetic data (only 41.93% correctly classified). Conversely, real sample classification improved to 81.77%. Following the TSTR evaluation 4.15 we see a noticeable increase (1.3456e-03) in ΔMAE compared to all weather features used.

The cloud-related features resulted in a performance pattern that combined elements of the two previous feature sets. The classifier achieved an Accuracy of 64.52% and an ROC of 70%. The confusion matrix shows that the classifier identified more real data correctly (71.48%), while the synthetic sample classification dropped to 57.55%. Surprisingly, in the TSTR evaluation 4.15, this model outperformed both the individual weather feature models and the model using all-weather features.

The combined temperature and humidity feature set performed unexpectedly worse than using temperature and humidity as separate features. Its behavior resembled that of the temperature-only model, particularly in the classifier evaluation, where real data proved to be the most challenging to classify. This feature combination resulted in the worst ΔMAE among all the feature combinations 4.15.

A summary of these results is presented in Table 4.16.

Table 4.16: Classifier Performance Metrics for Different Conditioning Data Feature Sets

| Feature Set | Accuracy (%) | ROC (%) | Synthetic Data (%) | Real Data (%) |
|---|---|---|---|---|
| Temperature Features | 64 | 71 | 73 | 55 |
| Humidity Features | **61.85** | **68** | 41.93 | 81.77 |
| Cloud Features | 64.52 | 70 | 57.55 | 71.48 |
| Temp. + Humidity | 63.93 | 70 | 73.44 | 54.43 |

**Impact of Statistical Features** For this experimental setup, the conditioning data was augmented with statistical features derived from the time series, identical to those used in the evaluation metrics: mean, standard deviation, skewness, kurtosis, minimum, and maximum values.

The results, summarized in Table 4.17, reveal a mixed impact from adding these statistical features. Improvements were observed in the alignment of standard deviation ($W = 0.0025$ vs. $W = 0.0103$) and kurtosis ($W = 0.8513$ vs. $W = 0.8912$), which achieved lower Wasserstein distances compared to the default model. However, the Wasserstein distances for the mean ($W = 0.0301$ vs. $W = 0.0070$) and minimum values ($W = 0.0057$ vs. $W = 0.0025$) increased, indicating a divergence from the real data for these specific statistics.

A similar pattern to the weather feature experiment was observed in the confusion matrices (Figure 4.34), where the inclusion of statistical features appeared to invert the classification strengths for real versus synthetic samples compared to the default model. In terms of discernibility, this configuration resulted in a decrease in AUC from 71% (default) to 67% and a reduction in accuracy from 63.74% (default) to 60.62%.

The TSTR evaluation revealed slightly worse performance when using statistical features as conditioning, compared to using only time features or the combination of time and weather features. The corresponding $\Delta$MAE values were $1.2132 \times 10^{-3}$ (statistical), $1.2093 \times 10^{-3}$ (time), and $1.2108 \times 10^{-3}$ (time + weather).

Table 4.17: Wasserstein Distances for Default (Time Features) vs. Time + Statistical Features Models (Lower $W$ is Better)

| Embedding | Mean($W$) | SD($W$) | Skew($W$) | Kurt($W$) | Min($W$) | Max($W$) |
|---|---|---|---|---|---|---|
| Default (Time features) | **0.0070** | 0.0103 | **0.2413** | 0.8912 | **0.0025** | 0.0494 |
| Time + Statistical features | 0.0301 | **0.0025** | 0.2500 | **0.8513** | 0.0057 | **0.0299** |

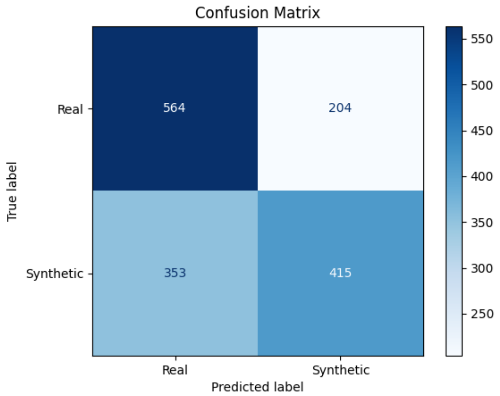



Figure 4.32: Confusion Matrix Default Model

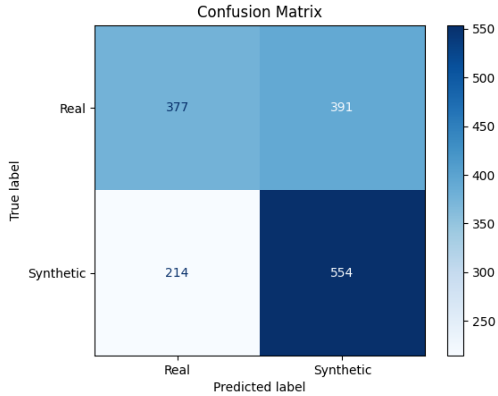



Figure 4.33: Confusion Matrix Time + Statistical Features Model

Figure 4.34: Comparison of Confusion Matrices for Default Model and Time + Statistical Features Model.

**Impact of Combined Weather and Statistical Features** This configuration involved conditioning the DDPM on a comprehensive set of features, combining all previously mentioned time, weather, and statistical variables.

As shown in Table 4.18, this combined approach yielded improvements in Wasserstein distances for standard deviation ($W = 0.0031$ vs. $W = 0.0103$), kurtosis ($W = 0.7855$ vs. $W = 0.8912$), and the maximum value ($W = 0.0322$ vs. $W = 0.0494$) when compared to the default time-feature conditioning. However, the Wasserstein distances for the mean ($W = 0.0303$ vs. $W = 0.0070$) and minimum values ($W = 0.0058$ vs. $W = 0.0025$) were higher, indicating less fidelity for these statistical properties.

Interestingly, the pronounced swapping pattern in classification performance observed in the confusion matrices with weather-only or statistical-only features (Figure 4.37) was less evident in this combined model. While the classification of real and synthetic samples did not mirror the default model as distinctly, the discernibility metrics showed a decrease in AUC from 71% (default) to 68% and a slight reduction in accuracy from 63.74% (default) to 62.17%.

Although this feature combination did not perform particularly well on the statistical metrics or the classifier evaluation, it achieved the best performance in the TSTR evaluation, with a $\Delta$MAE of $8.8662 \times 10^{-4}$, as shown in Table 4.20.

Table 4.18: Wasserstein Distances for Default (Time Features) vs. Time + Weather + Statistical Features Models (Lower $W$ is Better)

| **Embedding** | **Mean($W$)** | **SD($W$)** | **Skew($W$)** | **Kurt($W$)** | **Min($W$)** | **Max($W$)** |
|---|---|---|---|---|---|---|
| Default (Time features) | **0.0070** | 0.0103 | **0.2413** | 0.8912 | **0.0025** | 0.0494 |
| Time + Weather + Statistical feat. | 0.0303 | **0.0031** | 0.2544 | **0.7855** | 0.0058 | **0.0322** |

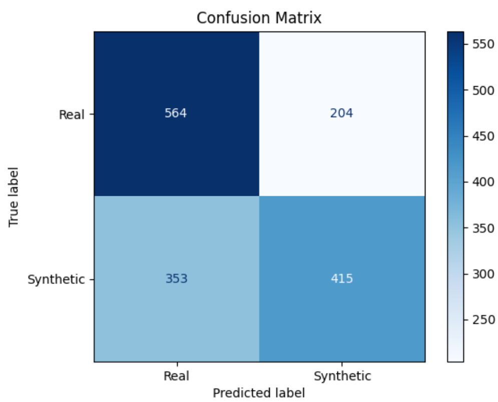



Figure 4.35: Confusion Matrix Default Model

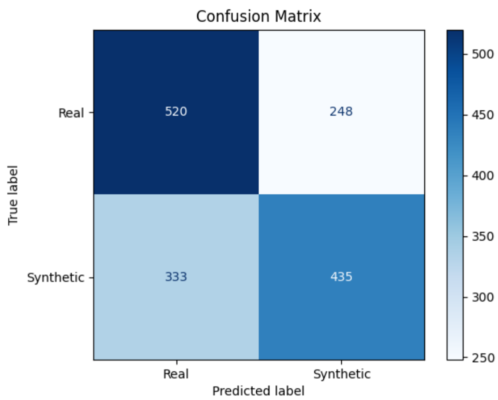



Figure 4.36: Confusion Matrix Time + Weather + Statistical Features Model

Figure 4.37: Comparison of Confusion Matrices for Default Model and Time + Weather + Statistical Features Model.

Table 4.19: Comparative Wasserstein Distances ($W$) for Different Conditioning Variables (Lower $W$ is Better)

| **Conditioning Type** | **Mean** | **SD** | **Skew** | **Kurt** | **Min** | **Max** |
|---|---|---|---|---|---|---|
| Default (Time Features) | **0.0070** | 0.0103 | **0.2413** | 0.8912 | **0.0025** | 0.0494 |
| Time + Weather | 0.0340 | 0.0038 | 0.3117 | 1.1112 | 0.0059 | 0.0331 |
| Time + Statistical | 0.0301 | **0.0025** | 0.2500 | 0.8513 | 0.0057 | **0.0299** |
| Time + Weather + Statistical | 0.0303 | 0.0031 | 0.2544 | **0.7855** | 0.0058 | 0.0322 |

Note: Bolded values indicate the best (lowest) Wasserstein distance for that specific statistical feature among the tested conditioning types.

Table 4.20: Comparison of ΔMAE Across Conditioning Types (Lower is Better)

| **Conditioning Type** | **ΔMAE** |
|---|---|
| Default (Time Features) | $1.2093 \times 10^{-3}$ |
| Time + Weather | $1.2108 \times 10^{-3}$ |
| Time + Statistical | $1.2132 \times 10^{-3}$ |
| Time + Weather + Statistical | **8.8662e-04** |

Note: Bolded value indicates the lowest MAE difference, suggesting the most realistic generation.

### RQ3: Comparative Hypothesis Testing of Promising Models

To evaluate the impact of the most promising input feature sets, we compare a model using combined weather and statistical features with a model using only the humidity feature. As the data does not follow a normal distribution, the Mann-Whitney U test is employed. The null hypothesis assumes no statistical difference between the two models, and a significance level of $\alpha = 0.05$ is used.

The results for MAE indicate no statistically significant difference between the models, with a p-value of 6.22e-02 and a Cohen's $d$ of 0.49. In contrast, the RMSE values do show a statistically significant difference, with a p-value of 3.34e-03 and a Cohen's $d$ of 0.90.

Table 4.21: Performance Comparison: Weather + Statistical Features vs Humidity Feature

| **Metric** | **Feature Set** | **Mean** | **Std** | **Cohen's d / p-value** |
|---|---|---|---|---|
| MAE | Weather + Statistical | 5.00e-02 | 0.0025 | 0.49 / 6.22e-02 |
| | Humidity Only | 4.91e-02 | 0.0011 | |
| RMSE | Weather + Statistical | 7.93e-02 | 0.0056 | 0.90 / 3.34e-03 |
| | Humidity Only | 7.53e-02 | 0.0028 | |

# 5

# DISCUSSION

This chapter provides a comprehensive discussion of the experimental results presented in Chapter 4, interpreting their significance in the context of the research questions posed in Chapter 3.1 and the existing literature reviewed in Chapter 2. The primary objective of this thesis is to investigate the efficacy of diffusion models for generating high-fidelity synthetic residential energy consumption data. This discussion will delve into the performance of various diffusion model architectures, the impact of different conditioning strategies, and the influence of diverse conditioning variables. By synthesizing these findings, this chapter aims to elucidate the strengths and limitations of current diffusion-based generative approaches for time series data and to highlight their implications for privacy-preserving data generation and downstream applications. The discussion is structured to address each research question sequentially, followed by an overall synthesis of the thesis's contributions and broader implications.

## 5.1. DISCUSSION OF RESEARCH QUESTION 1: OPTIMAL DIFFUSION MODEL ARCHITECTURE

This section addresses Research Question 1: "Which type of diffusion model is the most effective for generating synthetic data in residential energy consumption?" The comparative analysis of the TransFusion, 1D Conditional U-Net, and Physics-Informed models revealed distinct performance profiles, leading to the identification of the Transformer Encoder model (TransFusion) as the most effective architecture for this domain.

### 5.1.1. PERFORMANCE OF THE TRANSFUSION MODEL

The TransFusion model demonstrated superior capability in replicating a broad range of statistical characteristics of the real data. It achieved excellent fidelity for mean values, with a Wasserstein distance of 0.0070, and particularly for the minimum value distribution, showing a Wasserstein distance of 0.0025. Its replication of standard deviation (Wasserstein distance: 0.0103) and maximum values (Wasserstein distance: 0.0494) was also commendable. However, visual inspection of the generated distributions indicated a slight tendency for the model to smooth acute peaks in maximum values compared to the real data, while also showing a slight overestimation of density in certain ranges for the mean.

A key strength of TransFusion was its robust capture of temporal dependencies. Autocorrelation Function (ACF) distributions closely mirrored real data across various lags, indicating effective learning of underlying cyclical and seasonal patterns. Visual inspection of generated line plots further confirmed a strong resemblance to real time series, reinforcing its ability to capture inherent patterns.

The generated samples from TransFusion were significantly harder for a discriminative classifier to distinguish from real data, achieving an accuracy of 63.74% and an Area Under the ROC Curve (AUC) of 0.71. These metrics are notably closer to the desired 50% accuracy and 0.5 AUC, which would indicate perfect indistinguishability, suggesting higher fidelity and realism compared to the other models.

Regarding practical utility, the model exhibited exemplary performance in the Train on Synthetic, Test on Real (TSTR) evaluation. The forecasting model trained on synthetic data achieved a Mean Absolute Error (MAE) of 9.0058e-02, which was nearly identical to the MAE of 8.8849e-02 from the model trained on real data, resulting in a negligible difference ($\Delta$MAE = 1.2093e-03). This indicates that the synthetic data generated by TransFusion possesses comparable forecasting utility to real data, making it highly valuable for downstream applications. The primary trade-off for TransFusion's superior performance was its considerably longer training time, requiring 10 hours, 28 minutes, and 3 seconds for 5000 epochs.

### 5.1.2. Performance of the 1D Conditional U-Net Model

The 1D Conditional U-Net model showed mixed results. It replicated basic statistical features such as minimum value (W: 0.0130), standard deviation (W: 0.0354), and mean (W: 0.0387) reasonably well. However, it faced substantial challenges with higher-order moments, particularly kurtosis (W: 2.9663) and skewness (W: 0.4135), where the distributions showed significant divergence from the real data.

Regarding temporal dynamics, while the model began to capture seasonal and cyclical patterns, generated samples exhibited a higher level of noise and less distinct separations between cyclical periods compared to real data. Some deviations in autocorrelation performance were observed at longer lags, indicating a partial capture of temporal dependencies.

The synthetic data produced by the U-Net model had low fidelity and was easily distinguished from real data by the classifier, achieving an accuracy of 96.875% and an AUC of 1.0. This indicates significant room for improvement in achieving higher realism. In the TSTR evaluation, the synthetically-trained model achieved an MAE of 8.7024e-02, which was slightly worse than the real-trained model ($\Delta$MAE = 1.8244e-03). While this performance gap is larger than TransFusion's, a hypothesis test indicated no statistically significant difference in MAE between predictors trained on real and synthetic data (p=0.552889). A notable advantage of this model was its computational efficiency, training in approximately 49 minutes and 38 seconds. Attempts to increase its parameter count by using 4-layer and 5-layer versions did not yield accuracy improvements over the original three-layer model.

#### 5.1.3. Performance of the Physics-Informed Model

The Physics-Informed model exhibited significant deficiencies in learning the data's underlying patterns. It struggled profoundly to capture fundamental statistical properties, as evidenced by poor distributional overlap with real data and extremely high Wasserstein distances across most features, notably kurtosis (4.9106), skewness (0.9148), and maximum values (0.6591). Generated samples appeared significantly compressed in their dynamic range and overly smoothed, missing higher-frequency details and peak heights observed in the real data. Its synthetic data was readily distinguishable by the classifier, indicating very low fidelity. This model also scored the worst on the TSTR evaluation ($\Delta$MAE = 4.1155e-03) and incurred a long training time (13 hours, 10 minutes, and 31 seconds).

#### 5.1.4. Connection to the Literature

The superior performance of the TransFusion model aligns with the growing recognition of Transformer architectures' efficacy in sequence modeling. These models are particularly adept at capturing long-term temporal dependencies through their attention mechanisms. Sikder et al.'s [16] original work specifically highlighted TransFusion's strength in generating long-sequence, high-fidelity time series data, which is directly relevant to the 336-length sequences used in this study. The success observed here confirms that an attention-based denoising network is highly suitable for complex energy time series.

The mixed results of the 1D Conditional U-Net suggest that while U-Net architectures are powerful for tasks like image segmentation and have been adapted for time series, their convolutional nature might inherently struggle more with capturing the global, non-local dependencies critical for high-fidelity time series generation compared to attention-based models. Fu et al. [5] utilized a U-Net for energy data, but this comparative analysis indicates its limitations against a Transformer-based approach in this specific context.

The poor performance of the Physics-Informed model's baseline architecture suggests that its original efficacy might heavily rely on the integration of domain-specific physics constraints, which were not utilized in this study. This highlights a broader point: simply adopting a baseline architecture from another domain or stripping it of its key inductive biases may not yield strong generalizable performance. Furthermore, the choice of sequence length (336 time steps) may have exacerbated this issue. The model's LSTM backbone, while more robust than vanilla RNNs, remains inherently sequential—processing data one step at a time and relying on hidden and cell states to retain contextual information. As sequences grow longer, LSTMs often struggle to maintain relevant information across the entire input, leading to diluted or forgotten context. Although the addition of attention mechanisms can partially mitigate this, it may not be sufficient to fully address the challenge of modeling such long dependencies, especially when compared to architectures like Transformers that inherently model all time steps jointly through global self-attention.

The evaluation results highlight a nuanced relationship between statistical fidelity (measured by Wasserstein distances), perceived realism (assessed by a discriminative classifier), and practical utility (evaluated through TSTR). While TransFusion's synthetic data was not perfectly indistinguishable by the classifier (accuracy 63.74% is still above 50%), its performance in the TSTR evaluation was nearly identical to that of real data (negligible $\Delta$MAE).

Conversely, the 1D U-Net's synthetic data was easily distinguishable by the classifier (accuracy 96.875%), yet its TSTR performance, while worse than TransFusion, was still not statistically significantly different from the real-trained model (p-value 0.552889).

These results suggest limitations in the ability of commonly used statistical metrics to fully capture the fidelity of synthetic data. One plausible explanation lies in the multimodal nature of the real data distributions. When statistical comparisons are made across the entire distribution, metrics like the mean can mask important structural deficiencies. For example, if real data consists of two distinct modes (e.g., values clustered around 10 and 50), a generative model that only outputs values around their average (e.g., 30) may score well on certain statistical metrics despite failing to capture the underlying modes. This averaging effect leads to deceptively favorable scores even when fidelity—understood as the preservation of detailed data structure—is poor. Thus, statistical similarity may not be a sufficient proxy for perceptual or functional equivalence.

The TSTR metric, by contrast, directly assesses whether synthetic data preserves enough of the underlying data-generating process to be useful for downstream tasks. This is crucial for defining success in synthetic data generation. For applications where the primary goal is to train or augment machine learning models, TSTR should be prioritized over theoretical indistinguishability or perfect moment matching. These findings imply that "good enough" synthetic data for training purposes may still be detectable by a highly sensitive discriminator without compromising practical utility. Consequently, researchers should align evaluation metrics with intended application goals, steering model development toward practical performance rather than abstract similarity.

## 5.2. Discussion of Research Question 2: Impact of Conditioning Strategies

This section discusses Research Question 2: "How do different embedding mechanisms impact the performance of diffusion models in generating or augmenting customer energy consumption profiles?" The findings highlight the nuanced impact of various embedding techniques and the surprising outcome of different integration strategies.

### 5.2.1. Effect of Embedding Mechanisms

The study revealed that no single embedding mechanism was universally optimal across all statistical features and realism metrics, emphasizing the task-specific nature of optimal conditioning.

**Frequency-Domain Embeddings (FFT):** The FFT (magnitude) embedding emerged as a strong overall performer, achieving the lowest Wasserstein distances for mean (0.0050) and skewness (0.1027). It also performed very competitively for minimum (0.0043), standard deviation (0.0035), and maximum values (0.0287). Its classifier performance (accuracy: 56.97%, ROC AUC: 0.66) indicated good realism. The FFT (imaginary part) embedding yielded samples that were the most difficult for the discriminative classifier to distinguish from real data (accuracy: 57.16%, ROC AUC: 0.61), suggesting it produced the most realistic samples in that regard. It also showed good performance for kurtosis (0.5502) and minimum values (0.0044).

**Transformer Encoder Embedding:** Despite being trained for a significantly shorter duration (500 epochs compared to 5000 for others) due to computational constraints, the Transformer embedding demonstrated excellent performance in modeling higher-order moments, specifically kurtosis (W: 0.3681) and the distribution of maximum values (W: 0.0131). It also performed competitively for skewness (W: 0.1065) and minimum values (W: 0.0054). Its classifier performance (accuracy: 61.78%, ROC AUC: 0.66) indicated a reasonable degree of realism. Crucially, it achieved the best TSTR performance ($\Delta$MAE: 2.4775e-04) among all embedding mechanisms.

**STFT (Spectrogram) Embeddings (MLP vs. CNN Projector):** The comparison between STFT (MLP) and STFT (CNN) highlighted the critical impact of the projection network architecture. When using a simple MLP projector (STFT mlp), the model was best at replicating standard deviation (W: 0.0021) but consistently and significantly underperformed across all other statistical metrics, recording the highest Wasserstein distances for mean, skewness, and kurtosis (W: 2.7212). The resulting samples were easily identified by the classifier (accuracy: 83.66%, ROC AUC: 0.93), indicating very low realism and the worst TSTR performance ($\Delta$MAE: 1.0089e-01). In contrast, using a CNN projector (STFT cnn) yielded a dramatic improvement. It remained highly competitive for standard deviation (W: 0.0026) and demonstrated solid, mid-range performance across all other statistical features, including a respectable kurtosis distance (W: 0.7303). Crucially, its classifier performance (63.15% accuracy, 0.69 AUC) was vastly superior to the STFT (MLP) model and nearly identical to the Default (MLP) baseline, indicating substantially increased realism. It also achieved the second-best TSTR performance ($\Delta$MAE: 6.3624e-04).

**Default (MLP) Embedding:** This baseline embedding served as a solid reference, showing leading performance for minimum values (W: 0.0025) and remaining competitive for the mean (W: 0.0070). Its classifier performance (accuracy: 63.74%, ROC AUC: 0.71) indicated moderate realism.

### 5.2.2. Effect of Integration Strategies

The study compared the default concatenation method with a cross-attention mechanism for integrating conditioning data. The cross-attention model demonstrated improved performance (lower Wasserstein distances) over the default concatenation model for standard deviation (0.0057 vs. 0.0103), skewness (0.1855 vs. 0.2413), kurtosis (0.7434 vs. 0.8912), and notably for the maximum value (0.0126 vs. 0.0494). However, the default model maintained better fidelity for the mean value (0.0070 vs. 0.0168) and the minimum value (0.0025 vs. 0.0043). Temporal characteristics (ACF values) appeared largely similar for both models.

Critically, from the perspective of sample realism, the cross-attention mechanism resulted in data that was marginally more distinguishable by the classifier (accuracy: 65.10%, ROC AUC: 0.73) than the default concatenation approach (accuracy: 63.74%, ROC AUC: 0.71). This suggests that, in this specific aspect, the cross-attention mechanism did not enhance the realism of the generated samples. The cross-attention model also incurred an increased training time (11h 58m 13s) compared to the default concatenation (10h 28m 3s). Despite the mixed results in other metrics, the TSTR evaluation revealed no statistically significant difference between the baseline and cross-attention models (p-value of 0.064678).

## 5.3. Discussion of Research Question 3: Role of Conditioning Variables

This section explores Research Question 3: "What is the impact of the variables in the conditioning data in conditioned diffusion models?" The findings reveal that the choice and combination of conditioning variables significantly influence both the statistical fidelity and the perceived realism of the generated time series, often involving nuanced trade-offs.

### 5.3.1. Influence of Broad Conditioning Variable Categories

The investigation into different conditioning variable sets (default time, time + weather, time + statistical, time + weather + statistical) revealed a consistent pattern of trade-offs in statistical fidelity. While the default time-based conditioning excelled in replicating the mean (W: 0.0070) and minimum value distributions (W: 0.0025), alternative strategies offered benefits for other statistical moments. For instance, conditioning with 'Time + Statistical features' yielded the best or near-best results for standard deviation (W: 0.0025), kurtosis (W: 0.8513), and maximum value (W: 0.0299). Similarly, the combined 'Time + Weather + Statistical features' also showed improvements for standard deviation (W: 0.0031), kurtosis (W: 0.7855), and maximum values (W: 0.0322). However, these gains often came at the cost of slightly reduced fidelity for mean and minimum values compared to the default approach, highlighting a trade-off in optimizing for all statistical features simultaneously through conditioning.

From the perspective of perceived realism, assessed by a discriminative classifier's ability to distinguish synthetic samples from real ones, all broad alternative conditioning strategies (i.e., using the comprehensive weather feature set, statistical features, or their combination with time features) surpassed the default model. Lower classifier accuracy and ROC AUC scores, indicating greater difficulty in distinguishing samples and thus higher realism, were observed. Notably, the model conditioned on 'Time + Statistical features' demonstrated the highest realism in this comparison, achieving the lowest ROC AUC (67.00%) and accuracy (60.62%).

The Train Synthetic, Test Real (TSTR) evaluation provided a crucial measure of practical utility. Interestingly, despite strong performance on statistical metrics and classifier evaluation, the model using only statistical features performed the worst in the TSTR evaluation ($\Delta$MAE: 1.2132e-03). In contrast, the model combining statistical and weather features achieved the best performance in the TSTR evaluation, with a $\Delta$MAE of 8.8662×e-04, indicating its superior functional utility for forecasting tasks. Overall, the default, weather, and statistical models showed relatively similar performance in this evaluation.

### 5.3.2. Influence of Individual Weather Components

Further investigation into the impact of specific individual weather components (temperature, humidity, cloud) when combined with default time features provided more nuanced observations. Humidity features significantly enhanced realism, with classifier accuracy decreasing to 61.85% and ROC AUC to 68%. Critically, these generated samples were markedly more difficult for the classifier to identify as synthetic, with only 41.93% being correctly identified as such. Temperature features, in contrast, resulted in a slight increase in classifier accuracy (64%) and ROC AUC (71%) compared to the baseline (all weather features), suggesting a modest decrease in overall realism. For these samples, 73% of synthetic data was correctly identified. The use of cloud-related features yielded an accuracy of 64.52% and an ROC AUC of 70%, with 57.55% of synthetic samples being correctly identified. The observation that selective feature sets (e.g., temperature only) sometimes outperformed the inclusion of all weather features suggests that irrelevant or redundant features can introduce noise, hindering the model's performance.

### 5.3.3. Connection to the Literature

The findings regarding the impact of weather features align with existing literature that recognizes external environmental factors as significant covariates influencing energy consumption patterns [14]. However, this study extends that understanding by demonstrating that simply adding all available weather data is not necessarily optimal; selective inclusion based on relevance can yield better results, as suggested by previous research. This highlights the importance of domain knowledge-driven feature selection.

The strong performance of statistical features as conditioning data represents a form of "meta-conditioning." By explicitly providing the model with target distributional properties (mean, standard deviation, skewness, kurtosis, etc.), the generative process is directly guided to replicate the inherent statistical characteristics of the real data. This reinforces the idea that generative models should capture the underlying data distribution, and this approach provides a direct mechanism to achieve that. This extends the general concept of conditioning in diffusion models to a more abstract, distributional level.

# 6
# Conclusion

## 6.1. Summary of Key Findings

This thesis has demonstrated that diffusion models—when architecturally and conditionally optimized—hold substantial promise for generating high-quality synthetic residential energy consumption data. Rather than treating this as an isolated technical exploration, the findings provide a foundation for transformative applications in energy research, machine learning, and privacy-preserving data sharing. The work does not merely identify superior models; it reveals how specific design choices affect realism, utility, and statistical fidelity—highlighting important trade-offs and operational considerations for real-world use.

## 6.2. Insights from Research Questions

### 6.2.1. RQ1: Evaluating Optimal Diffusion Architectures

The Transformer-based TransFusion architecture emerged as the most effective model, excelling in statistical fidelity, temporal coherence, and downstream utility. However, its true impact lies in what it enables: credible synthetic data that can be used almost interchangeably with real data for forecasting applications. This bridges a key gap in the literature between theoretical model performance and functional deployment.

Moreover, the Transformer's limitations in capturing higher-order moments (like kurtosis) prompt an important shift in how we assess generative model success. It is no longer sufficient to target only visual or statistical similarity, the utility for downstream tasks (e.g., forecasting via TSTR evaluation) is a non-negotiable benchmark for future models. The implication is clear: functional realism must be prioritized over perceptual or metric-based fidelity alone.

### 6.2.2. RQ2: Effects of Conditioning Mechanisms

The study of embedding mechanisms and integration strategies yielded a critical insight: model design is deeply contextual. No single embedding approach universally dominated, and the architecture of the projector network (e.g., CNN vs. MLP for STFT) had a profound effect on both realism and fidelity.

This nuanced outcome shifts the discourse from selecting the "best" method to choosing the most suitable method for the desired output—whether it be realism, statistical fidelity, or functional utility. For practitioners, this encourages a tailored approach rather than defaulting to complexity (e.g., cross-attention) without a clear gain in utility.

### 6.2.3. RQ3: Influence of Conditioning Variables

Perhaps the most practically impactful finding lies in the effect of conditioning variables. Enriching the conditioning data, particularly with statistical features and specific weather components like humidity, directly improved realism, even if sometimes at the expense of precise statistical replication.

This indicates a pivotal truth: not all data is equally valuable for conditioning. Selective, domain-informed inclusion of features can drastically improve the believability and functional applicability of synthetic data. Importantly, this work shows that more data is not always better, and in some cases, irrelevant or noisy features degrade model performance.

For synthetic data in operational contexts (like smart grids or energy demand modeling), this supports a principled approach to feature selection, informed by both statistical and domain-specific heuristics.

## 6.3. Broader Implications and Contributions

This thesis advances the use of diffusion models for generating synthetic energy time series, enabling more reliable smart meter data for research, policy, and industry. High-quality synthetic data supports grid optimization, demand-side management, and renewable integration. Additionally, the evaluation of model components and conditioning strategies offers practical insights for applying generative AI to complex time series tasks.

The specific contributions of this research include:

- **Systematic Architectural Comparison:** Empirically established the Transformer Encoder (TransFusion) as the most effective diffusion model architecture for residential energy consumption data.

- **Comprehensive Evaluation of Conditioning Mechanisms:** Provided a detailed evaluation of various embedding mechanisms, highlighting the crucial role of the embedding network architecture (e.g., CNN for spectrograms) and the potential of Transformer embeddings.

- **Nuanced Analysis of Conditioning Variables:** Demonstrated the nuanced influence of different conditioning variables (weather, statistical, and their combinations) on statistical fidelity and perceived realism, identifying specific impactful variables like humidity.

- **Emphasis on Practical Utility (TSTR):** Reinforced the importance of the Train Synthetic, Test Real (TSTR) evaluation as a critical metric for assessing the practical utility of synthetic data for downstream machine learning tasks.

## 6.4. Study Limitations

While this research provides valuable insights, it is important to acknowledge its limitations, which in turn pave the way for future work:

- **Data Scope:** The study was conducted on a specific dataset of residential energy consumption. The findings, while robust within this context, may not generalize to other domains (e.g., commercial or industrial energy use) or different geographical regions with distinct climate patterns and consumption behaviors without further validation.
- **Computational Constraints:** As noted, some models, such as the Transformer embedding, were trained under significant time constraints. A more extensive training regimen could potentially alter their relative performance.
- **Architectural Scope:** While several key architectures were evaluated, the vast landscape of deep learning models was not exhaustively explored. Other promising architectures or hybrid models might offer further performance improvements.

## 6.5. Future Research Directions

Based on the findings and limitations of this thesis, several promising avenues for future research emerge:

- **Targeted Improvement of Higher-Order Moments:** Future work should focus on addressing the primary weakness of the Transformer model: its difficulty in capturing kurtosis. This could involve experimenting with novel loss functions that explicitly penalize discrepancies in higher-order moments or exploring architectural modifications designed to better model extreme value distributions.
- **Investigating the Realism-Fidelity Frontier:** The identified trade-off between statistical fidelity and perceived realism warrants further investigation. Future studies could explore multi-objective optimization techniques to find models that represent a "Pareto front" of optimal solutions balancing these two competing goals.
- **Advanced Architectural Exploration:** While this study identified the Transformer as the superior architecture, a logical next step involves exploring variants specifically optimized for time series analysis. Furthermore, investigating emergent architectures such as Kolmogorov-Arnold Networks (KANs) and extended Long Short-Term Memory (xLSTM), which have recently demonstrated significant potential in the time series domain, represents a valuable avenue for future research.
- **Incorporation of Diverse Meter Types:** Due to extensive missing data and time constraints, sub-datasets containing specific meter types (e.g., heat pumps, electric vehicles, solar PV) were not included in this study. Future research could explore the integration of these data sources to assess the impact of conditioning on device-specific variables—particularly in households where energy demand is more weather-dependent, such as those with heat pumps or solar panels.

# A

# DATASETS AND PRE-PROCESSING

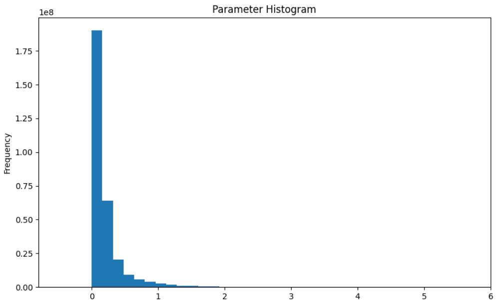



Figure A.1: Histogram Plot of the Parameter Column

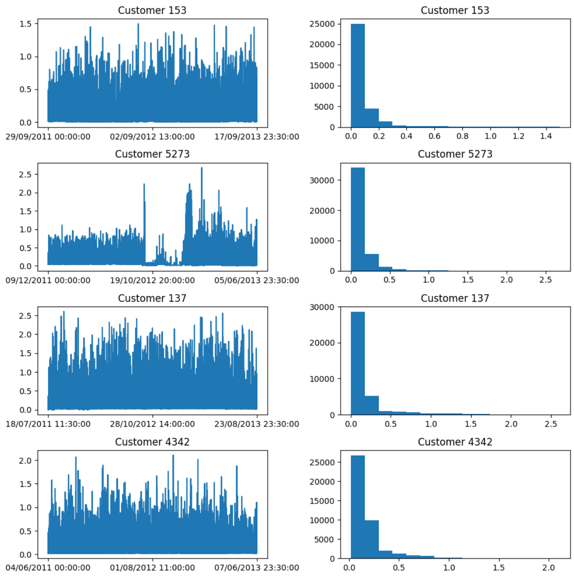



Figure A.2: Randomly Samples Customers

# Bibliography


[1] Seunghwan An and Jong-June Jeon. Distributional learning of variational autoencoder: Application to synthetic data generation. 1

[2] Martin Arjovsky and Léon Bottou. Towards principled methods for training generative adversarial networks. (arXiv:1701.04862), January 2017. doi: 10.48550/arXiv.1701.04862. URL http://arxiv.org/abs/1701.04862. arXiv:1701.04862 [stat]. 1, 8

[3] Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. Timevae: A variational auto-encoder for multivariate time series generation. (arXiv:2111.08095), December 2021. doi: 10.48550/arXiv.2111.08095. URL http://arxiv.org/abs/2111.08095. arXiv:2111.08095 [cs]. 8

[4] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. In *Advances in Neural Information Processing Systems*, volume 34, page 8780–8794. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper/2021/hash/49ad23d1ec9fa4bd8d77d02681df5cfa-Abstract.html. 8

[5] Chun Fu, Hussain Kazmi, Matias Quintana, and Clayton Miller. Creating synthetic energy meter data using conditional diffusion and building metadata. *Energy and Buildings*, 312:114216, June 2024. ISSN 0378-7788. doi: 10.1016/j.enbuild.2024.114216. iii, 1, 2, 9, 10, 16, 59

[6] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Commun. ACM*, 63(11):139–144, October 2020. ISSN 0001-0782. doi: 10.1145/3422622. 1, 8

[7] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, page 6840–6851. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html. 8

[8] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, November 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. 4, 5

[9] Hezheng Lin, Xing Cheng, Xiangyu Wu, and Dong Shen. Cat: Cross attention in vision transformer. In *2022 IEEE International Conference on Multimedia and Expo (ICME)*, page 1–6, July 2022. doi: 10.1109/ICME52920.2022.9859720. URL https://ieeexplore.ieee.org/abstract/document/9859720. 19

[10] Lequan Lin, Zhengkun Li, Ruikun Li, Xuliang Li, and Junbin Gao. Diffusion models for time-series applications: a survey. *Frontiers of Information Technology Electronic*

*Engineering*, 25(1):19–41, January 2024. ISSN 2095-9230. doi: 10.1631/FITEE.2300310. 8

[11] Zhi-Qiang Liu, Zhao Wang, Zhitao Zhao, Lianzhi Huo, Ping Tang, and Zheng Zhang. Bandpass alignment from sentinel-2 to gaofen-1 ard products with unet-induced tile-adaptive lookup tables. *Remote Sensing*, 15(1010):2563, January 2023. ISSN 2072-4292. doi: 10.3390/rs15102563. iii, 7

[12] Caspar Meijer and Lydia Y. Chen. The rise of diffusion models in time-series forecasting. (arXiv:2401.03006), January 2024. doi: 10.48550/arXiv.2401.03006. URL http://arxiv.org/abs/2401.03006. arXiv:2401.03006 [cs]. 14

[13] Frédéric Montet, Benjamin Pasquier, Beat Wolf, and Jean Hennebert. Enabling diffusion model for conditioned time series generation. *Engineering Proceedings*, 68(11): 25, 2024. ISSN 2673-4591. doi: 10.3390/engproc2024068025. 1, 10

[14] Rong Qu, Ruibing Kou, and Tianyi Zhang. The impact of weather variability on renewable energy consumption: Insights from explainable machine learning models. *Sustainability*, 17(11):87, January 2025. ISSN 2071-1050. doi: 10.3390/su17010087. 23, 51, 63

[15] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In Nassir Navab, Joachim Hornegger, William M. Wells, and Alejandro F. Frangi, editors, *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, page 234–241, Cham, 2015. Springer International Publishing. ISBN 978-3-319-24574-4. doi: 10.1007/978-3-319-24574-4_28. 4, 7

[16] Md Fahim Sikder, Resmi Ramachandranpillai, and Fredrik Heintz. Transfusion: Generating long, high fidelity time series using diffusion models with transformers. (arXiv:2307.12667), April 2024. doi: 10.48550/arXiv.2307.12667. URL http://arxiv.org/abs/2307.12667. arXiv:2307.12667 [cs]. iii, 2, 9, 15, 18, 59

[17] Kaleb E. Smith and Anthony O. Smith. Conditional gan for timeseries generation. (arXiv:2006.16477), June 2020. doi: 10.48550/arXiv.2006.16477. URL http://arxiv.org/abs/2006.16477. arXiv:2006.16477 [cs]. 8

[18] Hoang Thanh-Tung and Truyen Tran. Catastrophic forgetting and mode collapse in gans. In *2020 International Joint Conference on Neural Networks (IJCNN)*, page 1–10, July 2020. doi: 10.1109/IJCNN48605.2020.9207181. URL https://ieeexplore.ieee.org/abstract/document/9207181. 1, 8

[19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html. iii, 4, 6

[20] Jinsung Yoon, Daniel Jarrett, and Mihaela van der Schaar. Time-series generative adversarial networks. *Advances in Neural Information Processing*

*Systems*, 32, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/c9efe5f26cd17ba6216bbe2a7d26d490-Abstract.html?ref=https://githubhelp.com. 8

[21] Xinyu Yuan and Yan Qiao. Diffusion-ts: Interpretable diffusion for general time series generation. (arXiv:2403.01742), October 2024. doi: 10.48550/arXiv.2403.01742. URL http://arxiv.org/abs/2403.01742. arXiv:2403.01742 [cs]. 10

[22] Shaorong Zhang, Yuanbin Cheng, and Nanpeng Yu. Generating synthetic net load data with physics-informed diffusion model. (arXiv:2406.01913), June 2024. doi: 10.48550/arXiv.2406.01913. URL http://arxiv.org/abs/2406.01913. arXiv:2406.01913 [cs]. iii, 1, 2, 10, 16, 37